

설비 4대 분류와 측정의 3요소

설비를 네 갈래로 나누고, 컬럼 하나를 세 조각으로 분해합니다

회전기계 · 유압공압 · 열설비 · 반송 베어링과 진동

무엇을 · 어디서 · 얼마나 자주 분해능과 측정 상한 센서 카드

2026년 9월 1일 (화) · 30일차 · 70차시

교안 02-01 설비 4대 분류 · 실습 1 컬럼 목록 3요소 분해

국립금오공과대학교 컴퓨터공학부 박민호

교안 PART	다룬 범위	상태
01 설비 분류의 틀	동작 방식 분류 · 설비 4대 분류 지도	완주
02 현장의 설비들	회전기계 · 유압/공압 · 열설비와 반송 · 제철소 재배치	완주
03 측정의 3요소	물리량 · 설치 위치 · 샘플링 주기 · 실습 1	완주
04 측정값의 한계	분해능 · 측정 상한 · 오차 · 센서 카드	완주

이 책을 읽는 법

오늘의 두 축 · 뱃지 · 이전 회차와의 연결

1. 오늘은 축이 두 개입니다

29일차가 공정을 배웠다면, 30일차는 그 공정 안에 서 있는 **설비 한 대로** zoom합니다. 그리고 오전과 오후에 서로 다른 축을 하나씩 배웠습니다.

축		
질문	이 설비는 어떻게 움직이는가	이 컬럼은 무엇을 잰 숫자인가
도구	설비 4대 분류	측정의 3요소
결과물	어디가 먼저 닳는지 예측	물리량 · 설치 위치 · 샘플링 주기 표
한계 인식	대체 수단이 없는 설비가 더 중요	분해능 · 측정 상한보다 작거나 큰 변화는 없다

두 축은 따로 노는 것이 아닙니다. **설비를 분류해야 무엇을 잴지 정해지고, 무엇을 재는지 알아야 컬럼을 읽을 수 있습니다.** 오늘 실습은 정확히 그 다리를 건너는 연습이었습니다.

2. 항목 뱃지 읽는 법

본문의 각 항목 앞에는 **분류 뱃지**가 붙습니다. 뱃지가 문법 요소의 종류를 알려 줍니다.

개념 설비 · 측정 도메인 개념. 코드가 아니라 현장의 약속입니다.

메서드 객체 뒤에 점을 찍고 괄호를 붙여 호출합니다. `df.diff()` 처럼.

속성 객체 뒤에 점을 찍되 괄호가 없습니다. `df.columns` 처럼.

내장함수 앞에 점이 없습니다. `print()` · `list()` 처럼.

함수 라이브러리가 제공하는 함수입니다. `pd.read_csv()` · `pd.to_datetime()` 처럼.

패턴 한 줄짜리 문법이 아니라, 여러 도구를 엮은 **관용 표현**입니다.

개념

3초 판별법. ① 앞에 점(.)이 있나? 없으면 함수 · 내장함수입니다. ② 점이 있으면 괄호()가 붙나? 붙으면 **메서드**, 안 붙으면 **속성**입니다. 오늘 실습 노트북에 `columns()` 라고 썼다가 지운 흔적이 남아 있는데, 정확히 이 판별법으로 걸러지는 실수입니다.

3. 코드블록 범례

회색 배경에 왼쪽 파란 바가 있는 블록은 **파이썬 코드**입니다. 아래쪽 초록색 ▶ 줄은 그 코드를 **실제로 실행해 얻은 출력**입니다. 이 문서의 모든 숫자는 교안의 예상값이 아니라, 오늘 받은 CSV 두 개를 직접 돌려 확인한 값입니다.

예시

```
df_ts["timestamp"] = pd.to_datetime(df_ts["timestamp"])
print(df_ts["timestamp"].diff().value_counts(dropna=False))
```

```
▶ 0 days 00:01:00    119
▶ NaT                1
▶ Name: count, dtype: int64
```

4. 이전 회차와의 연결

회차	거기서 배운 것	오늘 어떻게 쓰었나
27일차	<code>to_datetime()</code> · <code>.diff()</code> · 시간 인덱스	실제 저장 간격 60초를 코드로 증명
28일차	<code>.diff()</code> 로 변화량 만들기 · 결측 드러내기	값이 변한 최소 폭 = 분해능 확인
29일차	공정·설비·라인 계층, 설비 태그 다섯 마디	태그 표기 두 방식 비교의 밑바탕
22일차	<code>.str</code> 접근자 · 조건 인덱싱	<code>.str.startswith()</code> 로 CASE A/B 분리

연결

29일차의 태그 다섯 마디(PL1-SNT-FAN-01-VIB)와 오늘의 짧은 태그(MTR01_VIB_RMS_H)는 **다른 체계가 아니라 같은 3요소를 다른 규칙으로 적은 것**입니다. 교안 30쪽이 이 둘을 나란히 놓고 비교합니다. 오늘 문서의 4-2 절이 그 지점입니다.

CONTENTS

목차

오늘 문서의 지도

장	제목	무엇을 배우나
CH 0	오늘의 지도	왜 이름이 아니라 동작 방식인가 · 4대 분류 지도 · 오늘의 데이터 두 개
CH 1	회전기계	베어링이라는 공통 약점 · 예지보전의 출발점 · 진동 반복 주기
CH 2	유압·공압	간헐 운전 · 명령값과 실제값 · 원인 지시형 데이터
CH 3	열설비와 반송	조작량이 먼저 움직인다 · 평균보다 분포 · 맥락 변수
CH 4	측정의 3요소	무엇을 · 어디서 · 얼마나 자주 · 태그 표기 두 방식
CH 5	실습 1	<code>.str.startswith()</code> · <code>.diff()</code> · 최소 변화 폭 · 블록 세기
CH 6	측정값의 한계	분해능 · 측정 상한 · 오차 · 센서 카드
부록 A	치트시트	오늘 나온 표를 한 장으로
부록 B	용어·오류 사전	워크시트 교정 · 노트북 코드 버그 · 헛갈리는 용어
부록 C	하이라이트와 다음 시간	오늘의 핵심 발견 · 31일차 예고

CH 0~3은 **현장 지식**, CH 4~6은 **데이터를 읽는 기술**입니다. 시간이 없다면 CH 4와 CH 6을 먼저 보시고, 그다음 CH 1로 돌아오시면 됩니다. CH 1의 진동 이야기가 CH 4의 "같은 진동을 나누는 세 단위"에서 다시 나오기 때문입니다.

주의

오늘 STT는 **147분(5세그먼트)**만 남아 있고, 마지막 발언이 "15분 정도 쉬는 시간을 가지고 돌아오도록 하겠습니다"로 끝납니다. 실습 노트북 저장 시각(14:33 / 14:57)을 보면 **쉬는 시간 이후 실습 진행 구간이 녹음에 없습니다**. 교안 42~47쪽과 노트북·워크시트가 모두 남아 있어 내용 복원은 문제없지만, 그 구간에서 강사님이 어디를 특히 강조하셨는지는 이 문서에서 추정으로 채웠습니다.

오늘의 지도

설비 분류의 틀 — 왜 이름이 아니라 동작 방식인가

0-1. 수천 개의 설비 이름을 네 개로 압축합니다

제철소 한 곳에 있는 설비의 종류는 수천 가지입니다. 이름을 다 외우는 것은 불가능하고, 외운다 해도 새 설비가 나오면 다시 막힙니다. 그래서 현장은 **이름이 아니라 동작 방식**으로 묶습니다.

개념

같은 방식으로 작동하는 설비는 같은 곳이 먼저 나뉘고, 같은 관측값으로 감시합니다. 이것이 4대 분류가 존재하는 이유의 전부입니다. 축이 도는 물건은 종류가 무엇이든 베어링이 먼저 상하고, 기름으로 힘을 내는 물건은 종류가 무엇이든 압력과 유온을 봅니다.

그래서 처음 보는 설비를 만났을 때의 첫 질문도 "이게 무슨 기계인가요"가 아니라 **"이건 돌아가나요, 미나요, 데우나요, 옮기나요"**가 됩니다. 강사님이 반복해서 강조한 지점입니다.



0-2. 설비 4대 분류 지도

오늘 전반부 전체가 이 표 한 장을 풀어 쓴 것입니다. 나머지 절은 각 행의 해설이라고 보셔도 됩니다.

분류	대표 설비	주요 관측값	운전 형태	데이터 특징
회전기계	모터·펌프·팬·압축기	진동·전류·베어링온도	연속 운전	끊김 없는 장기 추세
유압·공압	유압펌프·밸브·실린더	압력·유량·유온	간헐 동작	대기와 동작 구간 혼재
열설비	가열로·버너·열교환기	온도·연료유량·노압	느린 반응	긴 지연과 계절 영향
반송	컨베이어·크레인·대차	전류·속도·위치	조건 변화	위치·적재량에 따른 큰 변동

설비 적용

"데이터 특징" 열이 분석 방법을 결정합니다. 회전기계는 아무 구간이나 잘라도 비교가 되지만, 유압은 동작 구간을 먼저 골라내지 않으면 대기 중인 0을 평균에 섞게 됩니다. 반송은 적재량이 다르면 전류가 다른 것이 정상이라, 부하 보정 없이 비교하면 전부 이상으로 보입니다. 같은 `mean()` 한 줄이 분류에 따라 맞기도 하고 틀리기도 합니다.

0-3. 대수가 많다고 중요한 설비는 아닙니다

회전기계는 대수가 가장 많고 관련 자료도 가장 풍부해서 예지보전 기술이 제일 발달해 있습니다. 하지만 강사님은 여기서 한 번 브레이크를 거셨습니다.

주의

대체 수단이 없는 설비 한 대가 서는 손실이, 모터 열 대가 서는 손실보다 큽니다. 고로나 대형 크레인도 그렇습니다. 데이터가 많은 쪽과 중요한 쪽은 다를 수 있고, 예지보전 대상을 고를 때는 손실 크기를 함께 봐야 합니다.

대상 선정 기준은 하나 더 있습니다. **고장 전에 경고 시간이 있느냐**입니다. 베어링 마모처럼 진동·열·소리가 서서히 변하는 고장은 예측이 가능하지만, 전기 회로 단선이나 이물질 유입처럼 갑자기 터지는 사건은 예측 대상이 아닙니다.

미리 알 시간이 없는 고장은 예지보전의 대상이 아니라 보호 장치의 대상입니다.

선정 기준	확인 질문	아니라면
경고 시간	고장 전에 신호가 서서히 변하는가	보호 장치·차단기로 대응
손실 크기	한 대가 서면 라인이 서는가	우선순위를 낮춤
데이터 존재	이미 저장되고 있는 태그가 있는가	현장과 저장 주기부터 협의
동종 설비	비교할 같은 조건의 설비가 있는가	자기 자신의 과거와 비교

0-4. 오늘 다룬 데이터 두 개

후반부 실습에 쓰인 파일은 두 개입니다. 하나는 **사전**이고 하나는 **실제 기록**이라고 생각하시면 정확합니다.

파일	크기	성격	들어 있는 것
02-01_측정의_3요소_설비태그목록.csv	16행 8열	사전	태그명·설명·단위·사양 주기·범위·분해능·설치 위치
02-01_측정의_3요소_측정샘플.csv	120행 6열	기록	2026-03-02 08:00~09:59 · 5개 태그의 분당 값

두 파일의 관계가 오늘 실습의 핵심입니다. **사전에 적힌 사양이 실제 기록과 맞는지 대조**하는 것이 실습 1의 전부이고, 실제로 안 맞는 곳이 나옵니다.

오늘의 시작점

```
tags = pd.read_csv("02-01_측정의_3요소_설비태그목록.csv")
df = pd.read_csv("02-01_측정의_3요소_측정샘플.csv")

print(tags.shape, df.shape)
print("측정샘플 컬럼:", list(df.columns))
```

```
▶ (16, 8) (120, 6)
▶ 측정샘플 컬럼: ['timestamp', 'MTR01_VIB_RMS_H', 'MTR01_CURRENT', 'MTR01_TEMP',
▶               'HYD01_PRESS_IN', 'FUR01_TEMP_Z1']
```

개념

태그 목록에는 **16개** 태그가 있는데 측정 샘플에는 **5개**만 있습니다. **11개는 사양서에만 존재하고 실제 데이터는 못 받은 상태**입니다. 현장에서 늘 벌어지는 일이고, 교안 39쪽의 "현장과 합의할 데이터 요건"이 바로 이 간극을 좁히는 작업입니다.

0-5. 오늘 처음 나오는 용어 여덟 개

본문에 들어가기 전에 이름만 먼저 익혀 두면 읽기가 훨씬 수월합니다. 정확한 설명은 각 장에서 다시 나옵니다.

용어	한 줄 뜻	어디서 나오나
베어링	회전축을 받쳐 주는 부품. 회전기계의 공통 약점	CH 1
공동 현상	액체 속에 기포가 생겼다 터지는 현상. 펌프 흡입부에서 발생	CH 1
어큐레이터	유압 계통에서 압력을 저장했다 내보내는 부품	CH 2
작동유	유압 계통을 순환하며 힘을 전달하는 기름	CH 2
조작량	목표를 유지하기 위해 제어기가 바꾸는 값. 연료 유량 등	CH 3
맥락 변수	설비 상태가 아니라 다른 값의 의미를 정해 주는 정보	CH 3
샘플링 주기	값을 저장하는 시간 간격. 측정 3요소의 세 번째	CH 4
분해능	센서가 구분해 낼 수 있는 가장 작은 변화 폭	CH 6

CHAPTER 1

회전기계

축이 도는 모든 설비 — 예지보전이 가장 발달한 영역

1-1. 공통 구조와 공통 약점

회전기계는 **축의 회전으로 일을 만드는 설비군**입니다. 모터·펌프·팬·압축기·감속기가 여기 들어갑니다. 생김새는 전부 다르지만 공통 약점은 하나로 모입니다.

베어링입니다.

공통 조건	무슨 일이 벌어지나	결과
반복 운동	분당 수백~수천 회 같은 동작이 반복	작은 흠집이 매번 다시 눌림
마찰 접촉	구름체와 궤도면이 계속 닿음	마모 · 열 · 진동으로 확대

비유

구두 굽이 닳는 것과 같습니다. 한 걸음으로는 아무 일도 안 생기지만 **같은 자리에 같은 힘이 수만 번 반복되면** 반드시 티가 납니다. 회전기계의 좋은 점은 그 티가 **진동이라는 형태로 미리 나온다는 것**입니다.

1-2. 서서히 오는 고장과 갑자기 오는 고장

회전기계가 예지보전의 출발점이 된 이유는 대수가 많아서만이 아닙니다. **고장이 서서히 오기 때문**입니다.

구분	예시	신호	예지보전
서서히 진행	베어링 마모	진동·열·소리가 점진적으로 변화	대상
갑자기 발생	전기 회로 단선 · 이물질 유입	경고 시간이 짧거나 없음	대상 아님

개념

예지보전 대상 선정 기준은 **고장 전 경고 시간의 존재 여부**입니다. 아무리 비싼 설비라도 신호 없이 터지면 예측할 것이 없습니다. 반대로 값싼 모터라도 두 달에 걸쳐 진동이 올라간다면 훌륭한 대상입니다.

회전기계에서 상태를 읽는 물리량은 다섯 가지입니다. **진동 · 전류 · 온도 · 소리 · 윤활유의 상태**. 이 중 앞의 세 개가 오늘 실습 데이터에 그대로 들어 있습니다.

1-3. 모터 — 상태를 읽는 세 값

모터 한 대를 감시할 때 보는 값은 세 개로 정리됩니다. 여기서 중요한 것은 **취득 비용이 다르다**는 점입니다.

값	읽는 대상	어디서 얻나	비고
전류	부하 크기와 시동 상태	기존 제어 장치에서 취득	추가 설치 비용 없음
진동	축·베어링 상태	별도 센서 설치 필요	가장 이른 신호
온도	권선과 베어링의 과열 여부	권선 매입 측온저항체	결과 쪽 값

설비 적용

전류는 이미 있고 진동은 새로 달아야 합니다. 그래서 현장 프로젝트는 대개 "일단 전류부터 보자"로 시작합니다.

다만 전류는 부하가 바뀌면 같이 바뀌기 때문에, **부하 조건을 통제하지 않은 전류 비교는 의미가 없습니다.** 오늘 실습 데이터의 MTR01_CURRENT가 62~66 A 사이를 계속 오르내리는 것도 이상이 아니라 부하 변동입니다.

온도가 "결과 쪽 값"이라는 표현이 오늘 여러 번 나옵니다. **온도가 올랐다는 것은 이미 마찰이나 과부하가 한참 진행된 뒤**라는 뜻입니다. 원인을 직접 가리키지 않기 때문에, 온도만 보고 있으면 대응이 늦습니다.

1-4. 펌프에서 읽는 네 가지 신호

펌프는 모터에 "액체를 밀어낸다"는 임무가 추가된 설비입니다. 그래서 모터의 세 값에 **성능을 보는 값**이 붙습니다.

관측값	읽는 대상	열화 징후
토출압	압력 생성 성능	같은 조건에서 점진적 저하
유량	이송 성능	같은 회전 속도에서 감소
전류·진동	구동부 건강 상태	부하 증가 · 베어링 이상
흡입압·소음	공동 현상	기포 충격과 진동 증가

개념

공동 현상(cavitation) — 액체 속에 기포가 생겼다가 터지는 현상입니다. 흡입 쪽 압력이 너무 낮으면 물이 그 자리에서 끓듯 기포가 되고, 그 기포가 압력이 높은 곳에서 터지면서 금속 표면을 때립니다. 소리와 진동이 함께 올라가는 것이 특징입니다.

세 줄 중 "같은 조건에서" · "같은 회전 속도에서"라는 단서에 주목하실 필요가 있습니다. 성능 저하는 절대값으로 판정하는 것이 아니라 **조건을 맞춘 뒤의 변화**로 판정합니다. 조건이 다른 두 시점의 토출압을 그냥 빼면 열화가 아니라 운전 차이를 본 것입니다.

1-5. 이상 원인별 진동 반복 주기

오늘 전반부에서 가장 밀도가 높았던 대목입니다. **진동이 크다**는 것만으로는 원인을 못 찾지만, **회전 한 바퀴당 몇 번 덜컥거리는지**를 세면 원인이 갈립니다.

이상 종류	원인	반복 특성
불균형	회전체 무게 쏠림	회전 1회당 1번
정렬 불량	두 축의 어긋남	회전 1회당 2번
베어링 손상	궤도·구름체 마모	부위별 고유 간격
기어 손상	기어 이 파손	회전수와 이 개수의 조합

비유

불균형은 세탁기 탈수입니다. 빨래가 한쪽으로 쏠리면 통이 크게 흔들리고 소리가 납니다. 한 바퀴 돌 때마다 무거운 쪽이 한 번 지나가니까 **한 바퀴에 한 번** 덜컥거립니다. 정렬 불량은 축 두 개가 어긋나 있어 **한 바퀴에 두 번** 어긋난 지점을 지납니다.

베어링 손상과 기어 손상은 딱 떨어지는 정수 배가 아닙니다. 베어링은 **어느 부위(외륜·내륜·구름체)가 상했느냐에 따라 부딪히는 간격이 달라지고**, 기어는 **이의 개수와 회전수의 조합**으로 정해집니다. 그래서 정수 배가 아닌 주기가 나오면 그 자체가 단서가 됩니다.

주의

이 표를 읽으려면 **회전수(rpm)**를 알아야 합니다. 진동 값만 있고 회전수가 없으면 "1회당 몇 번"을 계산할 수 없습니다. 오늘 태그 목록에 MTR01_RPM이 들어 있는데 측정 샘플에는 빠져 있습니다. 실제 진단을 하려면 **반드시 같이 요청해야 하는 태그**입니다.

1-6. 회전기계 데이터의 네 종류

같은 진동이라도 **얼마나 자주 저장하느냐**에 따라 완전히 다른 데이터가 됩니다. 이 표가 CH 4의 샘플링 주기 이야기로 곧장 이어집니다.

종류	수집 주기	주요 용도	저장 부담
상태값	분 단위	장기 추세 감시	작음

종류	수집 주기	주요 용도	저장 부담
부하값	초 단위	조건 비교와 성능 평가	보통
진동 요약값	초·분 단위	이상 조기 감지	보통
진동 원신호	초당 수천 회	이상 손상 부위 특정	매우 큼

설비 적용

손상 부위를 특정하려면 원신호가 필요하지만, 원신호는 저장 부담이 압도적입니다. 그래서 현장은 평소 요약값만 저장하다가 이상 징후가 뜨면 그때 원신호를 켵니다. 오늘 실습 데이터의 MTR01_VIB_RMS_H가 바로 그 **요약값**입니다. RMS는 실효값, 즉 파형 전체를 숫자 하나로 줄인 값입니다.

1-7. 과거가 짧을 때 — 동종 설비 비교

예지보전의 정석은 "이 설비의 과거와 지금을 비교"하는 것입니다. 그런데 데이터를 모으기 시작한 지 두 달이면 비교할 과거가 없습니다. 그럴 때 쓰는 방법이 동종 설비 비교입니다.

같은 조건의 펌프 다섯 대 중 **유독 한 대만 진동이 높다면**, 그 한 대가 후보입니다. 과거 대신 **동료 설비를 기준선으로 삼는** 방식입니다.

확인 항목	왜 확인하나	안 맞으면
동일 부하·속도	조건이 다르면 값이 다른 게 정상	비교 자체가 성립하지 않음
동일 설치 환경	주변 온도·진동원이 값을 흔들	환경 차이를 이상으로 오판
데이터 종류	mm/s와 g는 다른 물리량	단위부터 통일해야 함
수집 주기	1초와 1분은 다른 것을 봄	요약 수준을 맞춰야 함

연결

27일차에 엔진 다섯 대의 센서를 나란히 놓고 비교했던 그 방식이 정확히 이것입니다. 그때는 "왜 여러 대를 같이 보나" 싶었지만, 오늘 이유가 나왔습니다. **과거가 짧아도 시작할 수 있는 유일한 방법**이기 때문입니다.

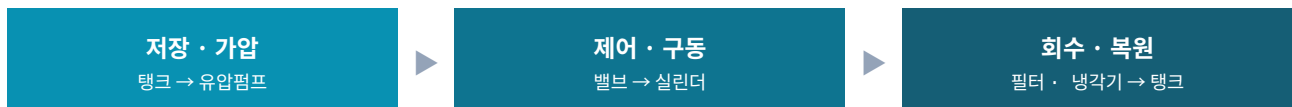
CHAPTER 2

유압·공압

기름으로 힘을 만드는 설비 — 간헐 운전 데이터의 함정

2-1. 유압 계통의 순환 흐름

유압은 기름에 압력을 걸어 힘으로 바꾸는 방식입니다. 큰 힘을 좁은 공간에서 만들 수 있어서 제철소의 압연 갭 조정, 랜스 승강, 코크스 압출기 같은 곳에 쓰입니다.



단계	무슨 일	여기서 나빠지면
저장·가압	탱크의 작동유를 유압펌프가 끌어 올려 가압	목표압 도달 시간이 길어짐
제어·구동	밸브가 방향·유량·압력을 조절, 실린더가 힘으로 변환	명령대로 안 움직임·유지압 하강
회수·복원	필터와 냉각기를 거쳐 탱크로 순환	필터 차압 증가·유온 상승

개념

순환입니다. 기름이 나갔다가 반드시 돌아옵니다. 그래서 계통 어느 한 곳이 나빠져도 결국 온도와 압력에 흔적이 남습니다. 문제는 그 흔적이 원인을 직접 가리키지 않는다는 점인데, 2-4절에서 이 이야기를 합니다.

2-2. 명령값과 실제값으로 구분하는 고장

유압 계통에는 밸브에 내린 명령이라는 값이 존재합니다. 회전기계에는 없는 축입니다. 이 덕분에 고장을 두 갈래로 깔끔하게 나눌 수 있습니다.

구분	증상	데이터에 나타나는 모습
동작 불량	밸브 명령은 존재하지만 위치·속도 응답이 부족	명령은 1인데 실제 위치가 안 따라옴
누설 불량	유지 압력 하강, 펌프 재가동 빈도 증가	정지 중인데도 압력이 계속 떨어짐

개념

명령과 실제 응답의 차이가 가장 이른 이상 신호입니다. 압력이 절대적으로 낮아지기 훨씬 전에, "명령한 만큼 안 움직인다"가 먼저 나타납니다. 그래서 유압 데이터를 받을 때는 **밸브 명령 신호를 같이 달라고 요청**하는 것이 핵심입니다.

누설 불량량의 두 번째 증상인 **"펌프 재가동 빈도 증가"**는 특히 계산하기 쉬운 지표입니다. 압력이 새면 시스템이 압력을 유지하려고 펌프를 더 자주 켵니다. **켜진 횟수를 하루 단위로 세는 것만으로도 추세가 보입니다.**

2-3. 유압 데이터의 첫 단계 — 동작 구간 선별

회전기계 데이터와 유압 데이터의 **결정적인 차이**가 여기 있습니다. 강사님이 이 대비를 정면으로 짚으셨습니다.

구분		
운전 형태	계속 돌아감 (연속)	필요할 때만 움직임 (간헐)
데이터 모양	끊김 없이 이어짐	대기 구간과 동작 구간이 섞임
아무 구간이나 잘라도?	된다 — 어차피 같은 사이클	안 된다 — 대기 중일 수 있다
첫 작업	바로 추세 분석	동작 구간부터 분리

자주 하는 실수

대기 구간이 섞인 유압 데이터에 그냥 `mean()`을 걸면, **압력이 0에 가까운 대기 시간이 평균을 끌어내립니다.** 설비가 멀쩡한데 "평균 압력이 떨어지고 있다"는 결론이 나옵니다. 실제로는 그날 작업량이 적었을 뿐입니다.

동작 구간을 나누는 기준은 두 가지입니다. **밸브 명령이 있으면 그것으로, 없으면 압력이나 유량이 문턱값을 넘는 구간으로 자릅니다.**

오늘 샘플에 적용

```
# 문턱값으로 동작 구간만 남기기 (개념 코드)
THRESH = 100          # bar, 대기/동작을 가르는 문턱
moving = df[df["HYD01_PRESS_IN"] > THRESH]

print("전체", len(df), "· 동작 구간", len(moving))
```

▶ 전체 120 · 동작 구간 120

주의

오늘 실습 데이터의 HYD01_PRESS_IN은 **최솟값이 151 bar**라서 대기 구간이 하나도 없습니다. 즉 이 두 시간은 **전부 동작 구간**입니다. 실제 현장 데이터라면 0에 가까운 값이 절반쯤 섞여 있을 텐데, 오늘 파일은 그 부분이 잘려 있는 셈입니다.

2-4. 관측값과 의심 원인 — 원인 지시형과 결과형

이 표가 CH 2에서 가장 중요합니다. 그냥 외우는 표가 아니라, **오른쪽 열이 하나로 좁혀지는지 여러 개로 벌어지는지**가 다릅니다.

관측값	악화 방향	의심 원인	성격
목표압 도달 시간	길어짐	펌프 성능 저하 · 누유	결과형
유지 압력	서서히 하강	밸브 · 실린더 내부 누설	결과형
유온	상승	내부 누설 · 냉각 성능 저하	결과형
필터 차압	증가	필터 막힘	원인 지시형
작동유 오염도	악화	입자 · 수분 오염	원인 지시형

개념

유온은 원인을 가리키지 않습니다. 여러 원인이 겹쳐서 나타나는 **결과 쪽 값**입니다. 모터에서 온도가 결과값이었던 것과 정확히 같습니다. 반면 **필터 차압이 커지면 필터가 막힌 것이 맞고, 오염도가 나빠지면 입자나 수분이 부품을 갉아먹고 있다는 뜻입니다.** 이 둘은 원인을 직접 짚어 줍니다.

실무 순서가 여기서 나옵니다. **결과형 값으로 "뭔가 나빠지고 있다"를 감지하고, 원인 지시형 값으로 "무엇이 나빠졌는지"를 확정합니다.** 결과형만 있으면 알람은 울리는데 조치를 못 하고, 원인 지시형만 있으면 감지가 늦습니다. 둘 다 필요합니다.

역할	해당 값	현장 요청 우선순위
조기 감지	유온 · 유지 압력 · 도달 시간	이미 저장되고 있을 확률 높음
원인 확정	필터 차압 · 작동유 오염도	별도 요청이 필요할 확률 높음

설비 적용

오늘 태그 목록의 HYD01_DP_FILTER가 바로 **필터 차압**인데, install_location이 "계산값 - 공급압력 - 후단압력"으로 적혀 있습니다. 즉 센서가 따로 있는 것이 아니라 HYD01_PRESS_IN 빼기 HYD01_PRESS_OUT으로 만든 파생값입니다. 4-5절에서 다시 다룹니다.

2-5. 유압 데이터 분석 절차 세 단계

교안이 제시한 순서입니다. 각 단계가 앞 단계 없이는 성립하지 않습니다.



단계	무엇을 하나	왜 필요한가
동작 구간 선별	밸브 명령으로 사이클의 시작과 종료를 구분	대기 구간이 섞이면 평균이 무의미
사이클 정렬	시작 시점을 맞춰 정상 동작 형상과 편차를 비교	사이클마다 시작 시각이 달라 그냥 겹치면 안 보임
온도 조건 통제	유온 구간별로 도달 시간과 유지 압력을 평가	기름은 차가우면 뻑뻑해 느려짐 — 열화가 아님

주의

STEP 3을 빼먹으면 아침마다 설비가 고장 난 것처럼 보입니다. 밤새 식은 작동유는 점도가 높아 목표압 도달이 실제로 느립니다. 유온을 조건으로 묶지 않고 도달 시간만 추세를 그리면, 계절과 시간대가 만들어 낸 패턴을 열화로 착각하게 됩니다.

열설비와 반응

데우는 설비와 옮기는 설비 — 그리고 맥락 변수

3-1. 열설비의 세 기능

열설비는 하는 일에 따라 세 갈래로 나뉩니다. 셋 다 "뜨겁다"는 점은 같지만 **감시해야 할 값이 다릅니다**.

기능	대표 설비	주 관심사
열 생성	가열로 · 소둔로 · 연소실 · 전기로	연료 유량 · 버너 상태
열 전달	열교환기 · 냉각탑 · 냉각수 계통	입출구 온도차 · 유량
열 저장	열풍로처럼 열을 축적한 뒤 방출	축열 · 방열 사이클 시간

연결

29일차에 나온 **열풍로**가 여기 "열 저장"에 해당합니다. 고로에 불어 넣을 공기를 미리 데워 두는 설비였습니다. 열을 만드는 것이 아니라 **모았다가 내보내는** 구조라, 사이클 시간이 짧아지면 축열 성능이 떨어졌다는 신호가 됩니다.

3-2. 열화는 결과값이 아니라 조작량에 먼저 나타납니다

오늘 배운 것 중 가장 실무적인 통찰입니다. **제어가 잘 되는 설비일수록 결과값은 정상으로 보입니다**.

구분	값	열화가 진행되면
제어 대상	노 내 온도	목표 범위에 그대로 유지 — 변화 없음
조작량	연료 유량	열손실을 보상하려 서서히 증가

개념

같은 온도를 만드는 데 늘어난 연료가 내화물 · 버너 열화의 단서입니다. 제어기가 목표를 지키려고 더 세게 때기 때문에 온도는 멀쩡합니다. 그러니 **온도만 보고 있으면 아무 일도 안 일어난 것처럼 보입니다**. 제어 루프가 있는 설비에서는 반드시 조작량을 같이 봐야 합니다.

이 원리는 열설비에만 해당하지 않습니다. **제어기가 붙어 있는 모든 설비**에 그대로 적용됩니다. 26일차에 다뤘던 이상 12건에서, 온도가 정상 범위 밖으로 나갔을 때 제어 출력이 이미 95.7~100%거나 0.6~5.6%로 **포화**되어 있었던 것이 정확히 같은 이야기였습니다. 조작량이 한계에 닿아서야 결과값이 무너진 것입니다.

제어 루프	제어 대상(결과값)	조작량(먼저 움직임)
가열로 온도 제어	노 내 온도	연료 유량
냉각수 온도 제어	냉각수 출구 온도	냉각탑 팬 출력 · 순환 유량
압력 제어	유지 압력	펌프 재가동 빈도
속도 제어	컨베이어 속도	모터 전류

3-3. 다지점 센서에서는 평균보다 분포

가열로에는 온도 센서가 여러 개 붙습니다. 오늘 태그 목록에도 FUR01_TEMP_Z1, FUR01_TEMP_Z2처럼 **존(zone)**별로 나뉘어 있습니다.

여기서 함정이 하나 있습니다. **버너 일부가 막혀도 전체 평균 온도는 정상 범위에 남습니다.**

보는 방식	무엇을 놓치나	무엇을 잡나
평균	반대 방향 편차가 서로 상쇄되어 소멸	전체 수준의 변화만
분포	—	특정 위치의 기울어짐과 편차 확대

비유

한 존이 30°C 낮아지고 다른 존이 30°C 높아지면 **평균은 그대로입니다**. 방은 한쪽이 얼고 한쪽이 타는데 온도계는 "적당함"이라고 말하는 셈입니다. 그래서 다지점 센서에서는 `mean()`이 아니라 `std()`와 **존 간 차이**를 봐야 합니다.

평균 대신 분포

```
# 존이 여러 개일 때 봐야 하는 값 (개념 코드)
zones = ["FUR01_TEMP_Z1", "FUR01_TEMP_Z2"]

df["zone_mean"] = df[zones].mean(axis=1)    # 놓치는 쪽
df["zone_std"]   = df[zones].std(axis=1)     # 잡아내는 쪽
df["zone_gap"]   = df[zones].max(axis=1) - df[zones].min(axis=1)
```

주의

오늘 측정 샘플에는 FUR01_TEMP_Z1만 있고 FUR01_TEMP_Z2는 없습니다. 즉 **존 간 편차를 볼 수가 없습니다**. 이것이 실습에서 "빈칸의 질문화"로 이어지는 대목입니다 — "2존 온도도 같이 받을 수 있습니까?"가 현장에 던져야 할 질문이 됩니다.

3-4. 열설비 데이터의 세 특징

특징	내용	분석에 미치는 영향
응답 지연	조작 시점과 결과 시점 사이에 여러 단계의 지연	원인-결과를 같은 시각으로 맞추면 안 됨
부드러운 변화	온도는 계단식이 아니라 서서히 오름	급격한 도약은 센서 이상 후보
구역 연동	여러 위치가 함께 움직임	한 지점만 다르면 국부 이상

개념

"급격한 온도 도약은 센서 이상 후보"가 오늘 강조된 대목입니다. 온도는 물리적으로 천천히 오를 수밖에 없습니다. 데이터가 한 스텝에 확 튀었다면 설비가 갑자기 뜨거워진 것이 아니라 센서나 통신이 이상할 확률이 훨씬 높습니다. 이상 탐지 알고리즘이 잡아낸 급변을 그대로 설비 이상으로 보고하면 안 되는 이유입니다.

응답 지연도 숫자로 감이 잡힙니다. 29일차에 나왔듯 **노 내 온도가 오르는 것보다 재료 속까지 열이 전달되는 데는 더 오래 걸립니다.** 그래서 "연료를 늘렸는데 왜 온도가 안 오르지"라는 판단을 너무 빨리 내리면 안 됩니다.

3-5. 반송 설비의 세 관리 특징

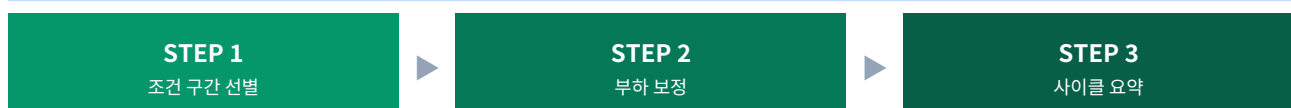
반송은 **옮기는 설비**입니다. 컨베이어·크레인·대차가 여기 들어갑니다. 29일차에 나온 소결광을 옮기는 **대차**도 반송입니다.

특징	내용	데이터 관점의 어려움
많은 부품 수	컨베이어 한 줄에 롤러 수백 개	전수 센싱이 불가능
다수 소모품	벨트·와이어로프·체인·롤러 교체	교체 이력이 데이터를 끊음
위치 의존성	설치 위치에 따라 정지 파급이 달라짐	같은 고장도 손실 크기가 다름

설비 적용

전수 센싱의 한계가 반송의 본질적 어려움입니다. 롤러 수백 개에 센서를 다 달 수는 없으니, **구동 모터 전류 하나로 수백 개의 상태를 추정**해야 합니다. 롤러 하나가 굳으면 전류가 아주 조금 올라가는데, 그 작은 차이가 부하 변동에 묻힙니다. 그래서 다음 절의 부하 보정이 필수가 됩니다.

3-6. 반송 데이터 분석 절차



단계	무엇을 하나	산출물
조건 구간 선별	같은 경로·적재량·경사 조건의 작업만 비교	비교 가능한 부분집합
부하 보정	적재 무게당 전류와 속도차로 상태 정규화	조건에 무관한 상태 지표
사이클 요약	왕복 작업별 최대 전류·소요 시간·위치 오차 추출	작업 1회 = 행 1개인 표

연결

유압의 3단계와 구조가 똑같습니다. ① **비교 가능한 구간만 남기고** ② **조건 차이를 없애고** ③ **사이클 단위로 요약한다**. 간헐 운전 설비를 다루는 일반 공식이라고 보셔도 됩니다. 회전기계만 이 3단계 없이 바로 추세를 볼 수 있습니다.

3-7. 해석을 완성하는 맥락 변수

오늘 마지막에 나온, 놓치기 쉬운 개념입니다. **설비 상태를 직접 재지 않지만 다른 센서값의 의미를 결정하는 정보**를 맥락 변수라고 합니다.

구분	예시	어디서 얻나
	전류 · 속도 · 온도 · 압력	설비의 센서
맥락 데이터	작업 종류 · 경로 · 부하 · 제품 조건 · 강종 · 규격	생산 시스템 (MES 등)

개념

전류 80 A라는 숫자만으로는 정상인지 알 수 없습니다. **빈 대차를 끌었을 때의 80 A는 이상이고, 만재 대차를 끌었을 때의 80 A는 정상**입니다. 같은 숫자가 맥락에 따라 정반대 해석이 됩니다. 그래서 맥락 데이터는 "있으면 좋은 것"이 아니라 **없으면 해석이 성립하지 않는 것**입니다.

실무적으로는 **센서 데이터는 설비 쪽 시스템에, 맥락 데이터는 생산 쪽 시스템에** 따로 있습니다. 두 시스템의 시각을 맞춰 붙이는 작업이 프로젝트 초반의 큰 일감이 됩니다.

3-8. 제철소 설비의 4대 분류 재배치

오늘 전반부의 결론입니다. **29일차에 배운 공정 순서와 오늘 배운 4대 분류**를 교차시킨 표입니다. 이 표를 채울 수 있으면 오늘 전반부는 소화한 것입니다.

분류	원료·제선	제강·연주	압연·유틸리티
회전기계	소결 배풍기 · 고로 송풍기	집진팬 · 냉각수 펌프	주모터 · 고압 펌프
유압·공압	코크스 압출기 · 개공기	랜스 승강 · 몰드 오실레이터	갭 조정 실린더

분류	원료·제선	제강·연주	압연·유틸리티
열설비	소결 점화로 · 열풍로	래들 예열 장치	가열로 · 소둔로
반송	스태커 · 벨트컨베이어	토페도 대차 · 크레인	롤러테이블 · 다운코일러

연결

29일차에 나온 이름들이 여기 다시 등장합니다. **고로 송풍기**는 회전기계, **열풍로**는 열설비, **토페도 대차**는 반송입니다. 어제는 "씻물을 옮기는 통"이었던 토페도 대차가, 오늘은 "**전류·속도·위치를 보고 조건 구간을 선별해야 하는 반송 설비**"가 되었습니다. 같은 설비를 보는 눈이 하나 더 생긴 셈입니다.

교안이 마지막에 남긴 질문 두 개가 이 문서의 나머지 절반으로 이어집니다.

질문 종류	묻는 것	오늘 어디서 답하나
구조 질문	어디가 먼저 닳고, 새고, 막히고, 흔들리는가	CH 1~3에서 분류별로
데이터 질문	어떤 값이 먼저 변하고, 어떤 조건을 함께 봐야 하는가	CH 4~6에서 측정의 3요소로

개념

산업이 바뀌어도 돌고, 밀고, 데우고, 옮기는 네 가지 물리 작동은 유지됩니다. 그래서 오늘 배운 틀은 제철소를 떠나도 그대로 쓰입니다. 반도체 팹의 진공 펌프도 회전기계고, 자동차 공장의 프레스도 유압입니다. "**이 산업은 처음인데요**"라는 말 대신 "**이건 도는 설비군요**"로 시작할 수 있게 됩니다.

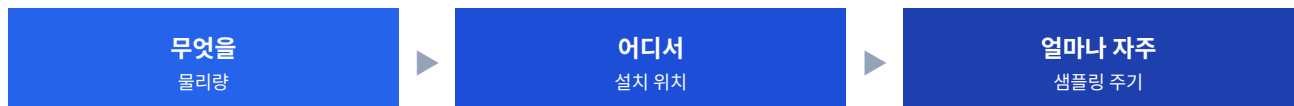
CHAPTER 4

측정의 3요소

무엇을 · 어디서 · 얼마나 자주 — 컬럼 하나를 세 조각으로

4-1. 모든 센서를 세 조각으로 분해합니다

후반부의 출발점입니다. 어떤 센서든 세 가지만 알면 그 컬럼이 무슨 숫자인지 설명할 수 있습니다.



요소	정하는 것	모르면 생기는 일
물리량	값이 반응하는 현상	단위조차 해석이 안 됨
설치 위치	신호의 크기	같은 고장인데 값이 달라 비교 불가
샘플링 주기	관찰 가능한 현상의 범위	있었던 사건을 없었다고 판정

교안의 한 문장으로 요약하면 이렇습니다. **"1번 모터에서 RMS_H(수평 방향 진동 속도)를 1초 간격으로 측정한 값"** — 이 한 문장 안에 위치·물리량·주기가 다 들어 있습니다. 컬럼 설명을 쓸 때 이 형식을 그대로 쓰시면 됩니다.

개념

세 요소는 **독립적이지 않습니다**. 물리량이 진동이면 주기가 빨라야 하고, 온도면 느려도 됩니다. 위치가 멀면 신호가 작아져서 분해능이 더 좋아야 합니다. **세 개를 한 세트로 같이 정하는 것이** 센서 설계이고, 데이터를 받은 쪽은 그 세트를 역으로 복원하는 셈입니다.

4-2. 같은 3요소, 표기 규칙만 다릅니다

29일차에 배운 다섯 마디 태그와 오늘 데이터의 태그는 생김새가 다릅니다. 그런데 **담고 있는 정보는 같습니다**.

표기	구조	어디서 쓰나
MTR01_VIB_RMS_H	설비 단위 · 장비ID + 물리량 + 방향	진동 분석기 · CMS
PL1-SNT-FAN-01-VIB	플랜트 계층 · 라인 + 공정 + 설비 + 호기 + 물리량	이력 서버

개념

설비 한 대를 보는 데이터와 제철소 전체 설비를 모은 데이터의 차이일 뿐입니다. 위치·물리량·측정 조건이라는 3 요소는 그대로입니다. 진동 분석기는 이미 어느 설비인지 알고 있으니 라인·공정을 안 적고, 전 공장 이력 서버는 태그가 수천 개라 라인부터 적어야 구분이 됩니다.

오늘 태그의 마디를 실제로 쪼개 보면 이렇습니다.

태그	설비	물리량	세부	방향/구역
MTR01_VIB_RMS_H	MTR01 (1번 모터)	VIB (진동)	RMS (실효값)	H (수평)
MTR01_VIB_RMS_A	MTR01	VIB	RMS	A (축방향)
HYD01_PRESS_IN	HYD01 (1번 유압)	PRESS (압력)	—	IN (공급/입구)
FUR01_TEMP_Z2	FUR01 (1번 가열로)	TEMP (온도)	—	Z2 (2존)

주의

약어는 **회사·시스템마다 다릅니다**. 오늘 배포된 용어 사전에 24개가 정리되어 있는데, 이런 사전이 없는 현장이 훨씬 많습니다. 그래서 **태그명을 100% 신뢰하면 안 되고**, 값의 범위와 움직임으로 교차 확인해야 합니다. TEMP 인데 값이 200을 넘으면 섭씨가 아니라 화씨일 수도 있습니다.

4-3. 물리량별로 잡아내는 상태 변화가 다릅니다

물리량	반응하는 상태 변화	대표 설비
진동	불평형 · 정렬불량 · 베어링 손상	회전기계
전류	부하 증가 · 마찰 증가	모터 구동 설비
온도	열 축적 · 냉각 불량	열설비
압력	누유 · 막힘 · 밀폐 실패	유압·공압

CH 0의 4대 분류 표와 나란히 놓으면 **같은 내용을 뒤집어 놓은 것**임을 알 수 있습니다. 앞에서는 "회전기계는 진동을 본다"였고, 여기서는 "진동은 회전기계를 본다"입니다. **설비에서 값으로 가는 길과 값에서 설비로 오는 길을 둘 다 익혀** 두면, 처음 보는 컬럼에서 설비를 역추적할 수 있습니다.

4-4. 같은 진동을 나누는 세 단위

"진동"이라고 다 같은 진동이 아닙니다. **단위가 다르면 잡아내는 이상이 다릅니다**. 오늘 후반부에서 강사님이 단위를 꼭 확인하라고 강조한 대목입니다.

단위	측정 물리량	잘 잡아내는 이상
mm/s	진동 속도	불평형 · 정렬 불량
g	진동 가속도	베어링 초기 손상
μm	진동 변위	축 흔들림

개념

셋은 서로 미분·적분 관계입니다. **변위 → (미분) → 속도 → (미분) → 가속도**. 가속도가 빠른 변화에 민감하기 때문에 **베어링의 미세한 충격을 잘 잡고**, 변위는 느리고 큰 움직임을 봐서 **축이 통째로 흔들리는 것을 잡습니다**. 속도는 그 중간이라 종합 지표로 가장 널리 쓰입니다.

자주 하는 실수

g는 **중력가속도 단위**이지 무게가 아닙니다. 실습 워크시트에서 MTR01_VIB_ACC의 물리량을 "무게"로 적은 답이 있었는데, 단위 g를 그램으로 읽은 것입니다. 태그 목록의 description에 **"1번 모터 비구동측 진동 가속도"**라고 명시되어 있으니 이쪽이 우선입니다. 부록 B에 교정해 두었습니다.

4-5. 직접 측정값과 파생값

컬럼이 있다고 해서 그 자리에 센서가 있는 것은 아닙니다. **다른 값으로 계산해서 만든 컬럼**이 섞여 있습니다.

구분	예시	만들어진 방식	오늘 태그
직접 측정	유압 압력	압력 센서의 직접 감지	HYD01_PRESS_IN
직접 측정	모터 전류	전류 센서의 직접 감지	MTR01_CURRENT
파생값	필터 차압	앞단 압력에서 뒷단 압력을 차감	HYD01_DP_FILTER
파생값	모터 전력	전압과 전류의 곱	MTR01_POWER

오늘 태그 목록의 `install_location` 열을 보면 파생값은 설치 위치 자리에 **"계산값 - ..."**이라고 적혀 있습니다. 이게 결정적인 단서입니다.

파생값 골라내기

설치 위치에 "계산값"이 들어간 태그 = 파생값

```
derived = tags[tags["install_location"].str.startswith("계산값")]
print(derived[["tag", "install_location"]].to_string(index=False))
```

- ▶ tag install_location
- ▶ MTR01_POWER 계산값 - 전압×전류 기반
- ▶ HYD01_DP_FILTER 계산값 - 공급압력 - 후단압력

주의

파생값은 원본 두 개의 오차를 함께 물려받습니다. 필터 차압은 공급압과 후단압의 차인데, 둘 다 분해능이 1 bar면 차압의 실질 분해능은 최악의 경우 2 bar가 됩니다. 차압이 3 bar 근처에서 움직이는 상황이라면 거의 노이즈만 보게 됩니다. 파생값을 쓸 때는 원본의 정밀도를 반드시 확인하셔야 합니다.

설비 적용

파생값에는 장점도 있습니다. 원인을 직접 가리킨다는 점입니다. 2-4절에서 봤듯 필터 차압은 "필터가 막혔다"를 곧장 말해 줍니다. 단독 센서보다 해석이 명확한 대신 정밀도를 잃는 거래인 셈입니다.

4-6. 설치 위치가 다른 값의 비교 원칙

오늘 가장 실무적인 경고가 여기 있습니다. 같은 이름의 컬럼이라고 같은 것을 잴 것이 아닙니다.

상황	판단 기준
같은 이름의 컬럼	설치 위치 일치 여부를 먼저 확인
위치가 다른 설비	절대값보다 설비 내부의 변화 추세를 비교
위치 정보 미확인	값의 크기 순서로 가설을 세운 뒤 현장 확인

개념

센서 위치가 진단 가능 여부를 정합니다. 같은 고장이라도 멀리 있는 센서는 값이 감쇠해서 정상처럼 보입니다.

구동측에서 잴 진동이 2.4 mm/s일 때 비구동측에서는 1.1 mm/s일 수 있고, 임계값 2.0을 걸어 두면 후자는 영원히 알람이 안 울립니다. 센서가 고장 난 것도, 설비가 멀쩡한 것도 아닙니다 — 위치가 다를 뿐입니다.

그래서 위치가 다른 절대값 비교는 성립하지 않습니다. 대신 각 설비 안에서의 변화 추세를 비교합니다. "A는 2.4, B는 1.1"이 아니라 "A는 두 달 새 +15%, B는 +42%"로 보는 것입니다. 이러면 위치 차이가 상쇄됩니다.

비교 방식	A 설비	B 설비	결론
절대값	2.4 mm/s	1.1 mm/s	A가 위험 — 틀린 결론일 수 있음
변화율	+15%	+42%	B가 위험 — 위치 차이에 강함

연결

28일차에 배운 `.pct_change()`가 여기서 쓰입니다. 그때는 "변화율을 왜 굳이 구하나" 싶었는데, **설치 위치가 다른 설비들을 한 화면에서 비교하려면 변화율밖에 답이 없습니다.** 절대값은 위치에 오염되어 있고 변화율은 그렇지 않습니다.

4-7. 샘플링 주기별 관찰 범위

세 번째 요소입니다. **얼마나 자주 저장하느냐가 무엇을 볼 수 있는지를 정합니다.**

샘플링 주기	주 용도	놓치는 현상
밀리초 단위	제어 · 진동 파형	더 빠른 미세 현상
1초 주기	설비 상태 감시	밀리초 스파이크
1분 주기	추세 감시	수십 초 변동
10분 주기	장기 추세	단시간 이상 대부분

개념

"놓치는 현상" 열이 핵심입니다. 1분 주기 데이터에서 30초짜리 스파이크를 찾는 것은 불가능합니다. 없어서 못 찾는 게 아니라 **처음부터 기록되지 않았기 때문**입니다. 이것을 모르면 "데이터를 다 봤는데 이상이 없었다"는 잘못된 결론을 내리게 됩니다.

그리고 물리량마다 **반응 속도가 다릅니다.** 교안에 구체적인 숫자가 나옵니다.

같은 부하 급증에 대해	반응까지 걸리는 시간	대응 가능 시간
전류	약 2분	가장 빠름
압력	약 6분	중간
온도	약 10분	가장 늦음

설비 적용

어떤 신호를 보느냐가 대응 시간을 정합니다. 온도만 보고 있으면 전류로 알 수 있었던 시점보다 8분 늦게 압니다. 8분이 별것 아닌 것 같아도, 그 사이 설비는 계속 과부하 상태였습니다. **느린 신호만 보면 대응이 늦습니다** — 그래서 알람을 설계할 때 빠른 물리량을 우선 확보합니다.

4-8. 타임스탬프로 확인하는 실제 저장 주기

사양서에 적힌 주기와 **실제로 저장된 간격이 다를 수 있습니다.** 확인 방법은 세 가지입니다.

확인 항목	방법	오늘 데이터에서
실제 간격 계산	연속 타임스탬프의 차이	전부 60초 (119개)
컬럼별 원주기	한 파일 안에서도 서로 다른 원래 주기	FUR01_TEMP_Z1은 사양 300초
반복값 해석	같은 값의 연속 반복에서 주기 맞춤 흔적	정확히 5행마다 변경

개념

한 파일 안에서 컬럼마다 원래 주기가 다를 수 있습니다. 저장 시스템이 가장 빠른 주기에 맞춰 표를 만들면, 느린 태그는 값이 안 바뀐 채 반복해서 채워집니다. 겉보기 간격은 60초인데 **실제 정보는 5분에 한 번**입니다. 이 흔적을 알아보는 것이 실습 1의 첫 번째 체크포인트입니다.

4-9. 현장이 저장 주기를 정하는 이유

"그냥 다 1초로 주세요"가 통하지 않는 이유입니다. 현장에는 나름의 사정이 있습니다.

이유	현장 배경	분석가의 대응
원래 목적	제어·감시 중심의 수집 설계	분석용 요건을 별도로 제시
저장 용량	수천 태그와 장기 보관 부담	핵심 태그만 선별 요청
통신 대역	제어 통신의 우선순위가 상위	전송 대상을 협의
물리량 특성	느린 값은 상대적 손실이 적음	빠른 물리량부터 우선 확보

주의

"원래 목적"이 가장 중요합니다. 현장의 데이터 수집 시스템은 **제어와 감시**를 위해 만들어졌지 분석을 위해 만들어진 것이 아닙니다. 그러니 분석가가 원하는 형태가 아니어도 그게 정상입니다. **불평이 아니라 요건 제시**로 접근해야 협의가 진행됩니다.

교안의 표현대로 **요청 범위와 물리적 근거를 좁힐수록 현장 협의 가능성이 높아집니다**. "전체 태그 1초"보다 "**이 3개 태그만 1초, 베어링 초기 손상이 1분 주기에서는 안 보이기 때문**"이 훨씬 잘 통합니다.

4-10. 데이터로 채우지 못한 빈칸을 질문으로 바꿉니다

오늘 실습의 마지막 단계이자, 이 수업 전체에서 반복되는 태도입니다. **모르는 칸을 비워 두거나 추측으로 메우지 않고, 질문 문장으로 바꿔 둡니다**.

확인 대상	현장 질문
설치 위치	어느 설비의 어느 지점에 설치된 센서인가

확인 대상	현장 질문
단위 정의	단위와 압력 기준은 무엇인가
저장 주기	원래 저장 주기는 얼마인가
값의 성격	직접 측정값과 파생값 중 어느 쪽인가
센서 이력	교체 · 교정 · 재설치 이력이 있는가

설비 적용

센서 이력이 특히 놓치기 쉽습니다. 센서를 교체하면 그 시점 전후로 값의 기준이 달라질 수 있는데, 데이터에는 아무 표시가 없습니다. 추세가 어느 날 갑자기 바뀌었다면 설비가 아니라 **센서가 바뀐 것**일 수 있습니다. 이력을 확보하지 못하면 그 시점을 경계로 나눠서 분석해야 합니다.

실습 1 — 컬럼 목록 3요소 분해

사양서와 실제 기록을 대조합니다

5-1. 실습 목표와 두 케이스

태그 목록과 측정 데이터로 **컬럼별 물리량·설치 위치·샘플링 주기** 표를 완성하는 것이 목표입니다. 케이스는 두 개로 나눠 진행했습니다.

케이스	대상	태그 개수	설비 분류
CASE A	MTR로 시작하는 태그	9개	회전기계
CASE B	HYD, FUR로 시작하는 태그	7개	유압 · 열설비

개념

케이스를 나눈 이유가 있습니다. **CASE A**는 태그명만 봐도 물리량이 거의 보이는 반면, **CASE B**는 공정 지식이 있어야 읽힙니다. VIB_RMS_H는 진동 실효값 수평이라고 짐작이 되지만, DP_FILTER가 필터 차압이고 그게 파생값이라는 건 CH 2를 거쳐야 알 수 있습니다.

5-2. 데이터 불러오기와 형태 확인

함수 `pd.read_csv()` CSV 파일을 DataFrame으로 읽기

정의 실험으로 구분된 텍스트 파일을 표 형태의 DataFrame으로 읽어 옵니다.

형태 `pd.read_csv(경로, encoding="utf-8")`

```
import pandas as pd

tags = pd.read_csv("02-01_측정의_3요소_설비태그목록.csv", encoding="utf-8")
df = pd.read_csv("02-01_측정의_3요소_측정샘플.csv", encoding="utf-8")

print(tags.shape, "\n · 결측", tags.isna().sum().sum())
print(df.shape, "\n · 결측", df.isna().sum().sum())
```

▶ (16, 8) · 결측 0
▶ (120, 6) · 결측 0

왜 쓰나 실습의 모든 단계가 이 두 표에서 출발합니다. **결측이 0**이라는 것을 먼저 확인해 두면, 뒤에서 나오는 "값이 안 변한다"가 결측 때문이 아님을 알 수 있습니다.

응용 한글 컬럼이나 한글 값이 들어간 CSV는 `encoding`을 명시하지 않으면 깨질 수 있습니다. 윈도우에서 저장된 파일이면 `encoding="cp949"`가 필요한 경우도 있습니다.

```
# 인코딩을 모를 때의 안전한 시도 순서
for enc in ["utf-8", "utf-8-sig", "cp949"]:
    try:
        tags = pd.read_csv(PATH, encoding=enc)
        print("성공:", enc); break
    except UnicodeDecodeError:
        print("실패:", enc)
```

▶ 성공: utf-8

주의

경로에 역슬래시를 쓸 때는 `"..\Data\파일.csv"`처럼 두 번 쓰거나 `r"..\Data\파일.csv"`처럼 raw 문자열로 써야 합니다.

속성 `.columns` 컬럼 이름 목록

정의 DataFrame의 컬럼 이름들을 담고 있는 속성입니다. 괄호를 붙이지 않습니다.

형태 `df.columns`

```
cols = list(df.columns)
print(cols)

▶ ['timestamp', 'MTR01_VIB_RMS_H', 'MTR01_CURRENT', 'MTR01_TEMP',
▶  'HYD01_PRESS_IN', 'FUR01_TEMP_Z1']
```

왜 쓰나 표를 처음 받았을 때 **무엇이 들어 있는지** 확인하는 첫 줄입니다. 오늘은 여기서 이미 "태그 목록에는 16개인데 여기는 5개뿐"이라는 발견이 나옵니다.

응용 태그 목록과 대조해서 **데이터가 안 온 태그**를 바로 뽑아낼 수 있습니다. 이 목록이 그대로 현장 요청 목록이 됩니다.

```
missing = [t for t in tags["tag"] if t not in df.columns]
print(len(missing), "개 태그가 사양서에만 있습니다")
for m in missing:
    print(" -", m)
```

```
▶ 11 개 태그가 사양서에만 있습니다
▶ - MTR01_VIB_RMS_V
▶ - MTR01_VIB_RMS_A
▶ - MTR01_VIB_ACC
▶ - MTR01_VOLT
▶ - MTR01_POWER
▶ - MTR01_RPM
▶ - HYD01_PRESS_OUT
▶ - HYD01_DP_FILTER
▶ - HYD01_FLOW
▶ - HYD01_OILTEMP
▶ - FUR01_TEMP_Z2
```

자주 하는 실수

`df.columns()`처럼 괄호를 붙이면 `TypeError`가 납니다. **속성이지 메서드가 아닙니다.** 배포된 실습 노트북에도 "columns 뒤에 ()를 지워주세요!"라는 주석이 남아 있습니다.

5-3. CASE A · CASE B — 접두어로 태그 나누기

메서드 `.str.startswith()` 문자열이 특정 글자로 시작하는지 검사

정의 문자열 Series의 각 값이 주어진 접두어로 시작하면 `True`, 아니면 `False`를 돌려줍니다.

형태 시리즈 `.str.startswith("접두어")` 또는 `.str.startswith(("A", "B"))`

```
# CASE A - 회전기계 계통
case_a = tags[tags["tag"].str.startswith("MTR")]

# CASE B - 유압·열설비 계통 (튜플로 여러 개 한 번에)
case_b = tags[tags["tag"].str.startswith(("HYD", "FUR"))]

print("CASE A", len(case_a), "개 · CASE B", len(case_b), "개")
```

```
▶ CASE A 9 개 · CASE B 7 개
```

왜 쓰나 태그 체계는 **앞자리에 설비를 적는 규칙**을 거의 항상 씁니다. 그래서 접두어 하나로 설비 계통이 갈립니다. 수천 개 태그가 있는 실제 현장에서 이 한 줄이 첫 필터가 됩니다.

응용 여러 접두어를 넘길 때는 **리스트가 아니라 튜플**이어야 합니다. 리스트를 넘기면 `TypeError`가 납니다. 계통별 개수를 한 번에 세면 태그 체계의 전체 구성이 보입니다.

```
prefix = tags["tag"].str.split("_").str[0]
print(prefix.value_counts().to_string())
```

```
▶ MTR01    9
▶ HYD01    5
▶ FUR01    2
```

개념

.str은 **접근자(accessor)**입니다. Series 자체에는 문자열 메서드가 없지만 .str을 거치면 각 원소에 문자열 메서드를 적용할 수 있게 됩니다. 22일차에 배운 .str.split()과 같은 계열입니다.

자주 하는 실수

tags["tag"].startswith("MTR")처럼 .str을 빼면 AttributeError가 납니다. Series에는 startswith가 없습니다.

메서드 .rename() 컬럼 이름 바꾸기

정의 columns 딕셔너리에 적힌 대로 컬럼 이름을 바꾼 **새 DataFrame**을 돌려줍니다.

형태 df.rename(columns={"원래이름": "새이름"})

```
df_renamed = tags.rename(columns={
    "tag": "태그명", "description": "물리",
    "unit": "단위", "install_location": "설치 위치",
    "sampling_sec": "사양 주기(초)",
})

result = df_renamed[["태그명", "물리", "단위", "설치 위치", "사양 주기(초)"]]
print(result.head(3).to_string(index=False))
```

```
▶   태그명      물리  단위   설치 위치  사양 주기(초)
▶ MTR01_VIB_RMS_H  1번 모터 구동측 수평 진동 실효값 mm/s  구동측 베어링 하우징 수평   60
▶ MTR01_VIB_RMS_V  1번 모터 구동측 수직 진동 실효값 mm/s  구동측 베어링 하우징 수직   60
```

왜 쓰나 워크시트의 칸 이름(**물리량 · 설치 위치 · 사양 주기**)에 맞춰 열 이름을 바꾸면, 그다음은 복사해서 옮기기만 하면 됩니다. 실습이 요구한 표를 코드로 바로 만드는 방법입니다.

응용 컬럼 선택과 조건 필터를 이어 붙이면 CASE B 표를 한 번에 뽑을 수 있습니다.

```
case_b_view = result[result["태그명"].str.startswith(("HYD", "FUR"))]
print(case_b_view.to_string(index=False))
```

```

>   태그명      물리 단위      설치 위치 사양 주기(초)
> HYD01_PRESS_IN   1번 유압 유닛 공급 압력 bar      펌프 토출 배관   60
> HYD01_PRESS_OUT 1번 유압 유닛 필터 후단 압력 bar      필터 후단 배관   60
> HYD01_DP_FILTER 1번 유압 필터 차압(계산값) bar 계산값 - 공급압력 - 후단압력 60
>   HYD01_FLOW 1번 유압 유닛 펌프 토출 유량 L/min      펌프 토출 배관 유량계   60
>   HYD01_OILTEMP 1번 유압 유닛 작동유 온도 degC      유압 탱크 내부   300
>   FUR01_TEMP_Z1 1번 가열로 1존 분위기 온도 degC      1존 천장 열전대   300
>   FUR01_TEMP_Z2 1번 가열로 2존 분위기 온도 degC      2존 천장 열전대   300

```

주의

.rename()은 원본을 바꾸지 않습니다. 결과를 변수에 담지 않으면 사라집니다. 원본을 바꾸려면 inplace=True 를 주거나 다시 대입해야 합니다.

5-4. 실제 저장 간격 확인

함수 pd.to_datetime() 문자열을 시간 타입으로 변환

정의 문자열로 읽힌 시각 컬럼을 datetime64 타입으로 바꿔, 뿔셈과 시간 연산이 가능하게 만듭니다.

형태 pd.to_datetime(시리즈)

```

print(df["timestamp"].dtype)          # 변환 전

df["timestamp"] = pd.to_datetime(df["timestamp"])
print(df["timestamp"].dtype)          # 변환 후
print(df["timestamp"].min(), "~", df["timestamp"].max())

> object
> datetime64[ns]
> 2026-03-02 08:00:00 ~ 2026-03-02 09:59:00

```

왜 쓰나 문자열끼리는 뿔 수 없습니다. 저장 간격을 구하려면 반드시 시간 타입으로 먼저 바꿔야 합니다. 27일차에 배운 첫 단추가 여기서 그대로 쓰입니다.

응용 변환한 뒤에는 .dt 접근자로 시·분을 꺼낼 수 있어, 시간대별 집계가 바로 가능합니다.


```
df["hour"] = df["timestamp"].dt.hour
print(df.groupby("hour")["MTR01_TEMP"].mean().round(3).to_string())
```

```
▶ hour
▶ 8      51.692
▶ 9      53.833
```

개념

오늘 데이터는 **2026-03-02 08:00부터 09:59까지 정확히 2시간, 120행**입니다. 데이터의 시간 범위를 먼저 확인해 두면 뒤의 모든 해석에 기준이 생깁니다.

메서드 `.diff()` 바로 앞 행과의 차이

정의 각 행에서 바로 앞 행의 값을 뺀 결과를 돌려줍니다. 첫 행은 뺄 대상이 없어 NaT/NaN이 됩니다.

형태 시리즈 `.diff()`

```
간격 = df["timestamp"].diff()
print(간격.value_counts(dropna=False).to_string())
```

```
▶ timestamp
▶ 0 days 00:01:00    119
▶ NaT                1
```

왜 쓰나 타임스탬프의 차이가 곧 실제 저장 간격입니다. 사양서에 뭐라고 적혀 있든, 실제로 저장된 간격은 이 한 줄로 확정됩니다. 60초가 119개, 첫 행의 NaT 1개 — **깔끔하게 1분 등간격**입니다.

응용 초 단위 숫자로 바꾸면 통계를 낼 수 있고, 결측 구간이 있으면 여기서 큰 값이 튀어나옵니다. 28일치의 결측 탐지가 정확히 이 방식이었습니다.

```
초 = 간격.dt.total_seconds()
print("최소", 초.min(), "· 최대", 초.max(), "· 고유값", 초.dropna().nunique())
```

```
▶ 최소 60.0 · 최대 60.0 · 고유값 1
```

개념

`.dt`는 `.str`과 같은 **접근자**입니다. 시간 타입 Series에 `.dt`를 붙여야 `total_seconds()`, `hour`, `date` 같은 시간 전용 기능을 쓸 수 있습니다.

자주 하는 실수

`to_datetime()`을 건너뛰고 `.diff()`를 부르면 문자열끼리 빼려다 오류가 납니다. **순서가 고정**입니다.

5-5. 값의 크기와 움직임 관찰

메서드 `.agg()` 여러 통계를 한 번에

정의 집계 함수 이름을 리스트로 넘기면, 각 컬럼에 대해 그 통계들을 한 표로 계산합니다.

형태 `df[컬럼목록].agg(["min", "max", "mean", "std"])`

```
cols = ["MTR01_VIB_RMS_H", "MTR01_CURRENT", "MTR01_TEMP",
        "HYD01_PRESS_IN", "FUR01_TEMP_Z1"]
```

```
print(df[cols].agg(["min", "max", "mean", "std"]).round(3).to_string())
```

	MTR01_VIB_RMS_H	MTR01_CURRENT	MTR01_TEMP	HYD01_PRESS_IN	FUR01_TEMP_Z1
▶ min	1.700	62.000	48.000	151.000	1180.000
▶ max	2.400	66.000	54.000	210.000	1191.500
▶ mean	2.020	63.750	52.762	159.025	1185.750
▶ std	0.187	1.083	1.553	18.622	3.476

왜 쓰나 다섯 컬럼의 성격이 이 표 하나로 드러납니다. HYD01_PRESS_IN의 표준편차 18.622가 유독 큰데, 최댓값 210이 평균 159에서 멀리 떨어져 있기 때문입니다. 여기서부터 "왜 210에 몰려 있지"라는 의심이 시작됩니다.

응용 태그 목록의 `range_min/range_max`와 나란히 놓으면 **측정 범위의 어디쯤을 쓰고 있는지** 바로 보입니다. 범위를 거의 안 쓰는 센서는 분해능을 낭비하고 있는 것입니다.

```
spec = tags.set_index("tag").loc[cols, ["range_min", "range_max"]]
spec["실제최소"] = df[cols].min().values
spec["실제최대"] = df[cols].max().values
spec["범위사용률"] = ((spec["실제최대"] - spec["실제최소"]) /
                     (spec["range_max"] - spec["range_min"])) * 100).round(1)
print(spec.to_string())
```

	range_min	range_max	실제최소	실제최대	범위사용률
▶ MTR01_VIB_RMS_H	0	25	1.7	2.4	2.8
▶ MTR01_CURRENT	0	120	62.0	66.0	3.3
▶ MTR01_TEMP	0	180	48.0	54.0	3.3
▶ HYD01_PRESS_IN	0	210	151.0	210.0	28.1
▶ FUR01_TEMP_Z1	0	1300	1180.0	1191.5	0.9

개념

범위 사용률이 1~3%라는 것은 **정상 운전 중이라 값이 거의 안 움직인다**는 뜻이지, 센서가 잘못된 것이 아닙니다. 측정 범위는 이상 상황까지 담을 수 있게 넓게 잡습니다.

패턴 값이 변한 최소 폭 분해능을 데이터에서 역산

정의 `.diff().abs()`로 변화 폭을 만든 뒤, **0(변화 없음)을 제외하고** 최솟값을 구합니다.

형태 `시리즈.diff().abs().replace(0, pd.NA).min()`

```
for col in cols:
    min_step = df[col].diff().abs().replace(0, pd.NA).min()
    res = tags.set_index("tag")["resolution"].get(col)
    print(f"{col:16} 최소 변화폭 {round(min_step, 4):>6} | 분해능 {res}")
```

```
▶ MTR01_VIB_RMS_H   최소 변화폭   0.1 | 분해능 0.1
▶ MTR01_CURRENT     최소 변화폭   0.5 | 분해능 0.5
▶ MTR01_TEMP        최소 변화폭   0.5 | 분해능 0.5
▶ HYD01_PRESS_IN    최소 변화폭   1.0 | 분해능 1.0
▶ FUR01_TEMP_Z1     최소 변화폭   0.5 | 분해능 0.5
```

왜 쓰나 다섯 개가 전부 사양서의 분해능과 정확히 일치합니다. 사양서를 안 보고도 데이터만으로 분해능을 알아낼 수 있다는 뜻입니다. 반대로 말하면, **분해능보다 작은 변화는 데이터에 아예 없습니다.**

응용 `.replace(0, pd.NA)`가 핵심입니다. 이걸 빼면 값이 안 바뀐 행의 0이 최솟값이 되어 항상 0.0이 나옵니다. 아래는 그 차이를 나란히 본 것입니다.

```
for col in cols[:3]:
    with0 = df[col].diff().abs().min()
    no0    = df[col].diff().abs().replace(0, pd.NA).min()
    print(f"{col:16} 0 포함 {with0} | 0 제외 {round(no0, 4)}")
```

```
▶ MTR01_VIB_RMS_H   0 포함 0.0 | 0 제외 0.1
▶ MTR01_CURRENT     0 포함 0.5 | 0 제외 0.5
▶ MTR01_TEMP        0 포함 0.0 | 0 제외 0.5
```

자주 하는 실수

MTR01_VIB_RMS_H의 실제 계산 결과는 0.099999999999999964입니다. **부동소수점 오차**라 `round()` 없이 비교하면 `0.1 == min_step`이 `False`가 됩니다. 사양과 대조할 때는 반드시 반올림하시거나 `abs(a-b) < 1e-9`로 비교하세요.

패턴 연속 반복 블록 세기 같은 값이 몇 번 반복되는지

정의 `(s != s.shift()).cumsum()`으로 **값이 바뀔 때마다 번호가 증가하는 블록 ID**를 만들고, 블록별 크기를 셉니다.

형태 `block = (s != s.shift()).cumsum(); df.groupby(block)[col].size().max()`

```
for col in cols:
    block = (df[col] != df[col].shift()).cumsum()
    max_repeat = df.groupby(block)[col].size().max()
    print(f"{col:16} 최대 연속 반복 {max_repeat:2}회 | 고유값 {df[col].nunique():2}개")
```

- ▶ MTR01_VIB_RMS_H 최대 연속 반복 2회 | 고유값 8개
- ▶ MTR01_CURRENT 최대 연속 반복 1회 | 고유값 9개
- ▶ MTR01_TEMP 최대 연속 반복 40회 | 고유값 13개
- ▶ HYD01_PRESS_IN 최대 연속 반복 7회 | 고유값 5개
- ▶ FUR01_TEMP_Z1 최대 연속 반복 5회 | 고유값 24개

왜 쓰나 노트북 주석에 "행간 데이터 변화 중 최소 폭은 메서드로 알 수 없습니다"라고 적혀 있듯, 반복 횟수를 세는 기본 메서드가 따로 없습니다. 이 블록 기법이 정식 해법입니다.

응용 FUR01_TEMP_Z1의 최대 연속 반복이 **정확히 5회**인 것이 결정적 단서입니다. 값이 바뀌는 시각만 뽑아 보면 5분 간격으로 딱 떨어집니다.

```
chg = df[df["FUR01_TEMP_Z1"].diff() != 0]["timestamp"]
print(chg.dt.strftime("%H:%M").tolist()[:8])
print("변경 횟수", len(chg) - 1, "회 · 사양 주기",
      tags.set_index("tag").loc["FUR01_TEMP_Z1", "sampling_sec"], "초")
```

- ▶ ['08:00', '08:05', '08:10', '08:15', '08:20', '08:25', '08:30', '08:35']
- ▶ 변경 횟수 23 회 · 사양 주기 300 초

개념

(s != s.shift())는 "앞 값과 다른가"를 묻는 불리언 Series이고, .cumsum()은 True를 1로 더해 갑니다. 그래서 **값이 바뀔 때만 번호가 하나 올라가는** 블록 ID가 됩니다. 결측 구간 찾기, 상태 유지 시간 재기 등에 두루 쓰이는 관용 표현입니다.

5-6. 실습 1 체크포인트 세 개

교안 47쪽이 요구한 세 가지를 오늘 데이터로 확인한 결과입니다. **셋 다 실제로 나왔습니다.**

① 주기 불일치 — 사양과 실재가 다른 컬럼

태그	사양 주기	실제 저장 간격	실제 값 변경 주기	판정
MTR01_VIB_RMS_H	60초	60초	60초	일치
MTR01_CURRENT	60초	60초	60초	일치
MTR01_TEMP	60초	60초	불규칙	일치

태그	사양 주기	실제 저장 간격	실제 값 변경 주기	판정
HYD01_PRESS_IN	60초	60초	60초	일치
FUR01_TEMP_Z1	300초	60초	300초 (5행마다)	불일치

개념

FUR01_TEMP_Z1은 저장 간격은 60초지만 실제 정보는 300초에 한 번입니다. 5분에 한 번 읽은 값을 그 사이 5행에 그대로 복사해 채운 것입니다. 이 컬럼으로 1분 단위 변화를 분석하면 전부 허구입니다. .diff()를 걸면 5행 중 4행은 무조건 0이 나옵니다. 리샘플링해서 5분 단위로 낮춰 놓고 봐야 맞습니다.

② 계단형 변화와 상한 반복

교안 45쪽이 표시하라고 한 두 가지입니다. 최소 변화 폭이 일정한 컬럼과 동일한 최댓값이 반복되는 컬럼입니다.

태그	최소 변화 폭	최댓값	range_max	상한 도달	해석
MTR01_TEMP	0.5	54.0	180	아님	실제 안정화
HYD01_PRESS_IN	1.0	210.0	210	도달 · 14회	상한 포화 의심
FUR01_TEMP_Z1	0.5	1191.5	1300	아님	주기 맞춤 반복

주의

MTR01_TEMP와 HYD01_PRESS_IN은 겉보기가 비슷하지만 성격이 완전히 다릅니다. 둘 다 최댓값이 오래 반복되지만, MTR01_TEMP의 54.0은 측정 상한(180) 근처가 아니라 실제로 온도가 안정된 것이고, HYD01_PRESS_IN의 210.0은 range_max와 정확히 같아서 상한에 걸린 것입니다. 반복만 보고 판정하면 안 되고 반드시 range_max와 대조해야 합니다.

체크포인트 ②

상한 반복 여부 판정

```
for col in cols:
    rmax = tags.set_index("tag").loc[col, "range_max"]
    hit = (df[col] == rmax).sum()

    print(f"{col:16} 최댓값 {df[col].max():>7} | range_max {rmax:>6} | 상한 도달 {hit:2}회")
```

```
▶ MTR01_VIB_RMS_H   최댓값   2.4 | range_max      25 | 상한 도달 0회
▶ MTR01_CURRENT     최댓값  66.0 | range_max     120 | 상한 도달 0회
▶ MTR01_TEMP        최댓값  54.0 | range_max     180 | 상한 도달 0회
▶ HYD01_PRESS_IN    최댓값 210.0 | range_max     210 | 상한 도달 14회
▶ FUR01_TEMP_Z1     최댓값 1191.5 | range_max    1300 | 상한 도달 0회
```

210이 나온 시각을 뽑아 보면 **연속된 두 덩어리**입니다. 흩어져 있는 것이 아니라 **7분씩 두 번** 몰려 있습니다.

구간	시각	길이	해석 가설
1차	08:03 ~ 08:09	7분	실제 고압 작업 또는 센서 포화
2차	09:10 ~ 09:16	7분	같은 패턴의 반복

개념

보이는 패턴과 해석 가설을 분리해서 적는 것이 교안 45쪽의 세 번째 요구사항이었습니다. 여기서 확실한 사실은 **"210이 7분씩 두 번 연속으로 나왔고, 210은 range_max와 같다"까지입니다. "센서가 포화됐다"는 가설이고, "실제로 210 이상 올라갔는데 잘린 것"인지 "정말 210에서 유지된 것"인지는 이 데이터만으로는 구분할 수 없습니다. 현장 확인이 필요한 항목입니다.**

③ 남은 질문 목록

데이터로 채우지 못한 칸을 그대로 질문 문장으로 옮겼습니다. 이것이 실습 1의 최종 산출물입니다.

대상	확인 질문	근거
FUR01_TEMP_Z1	원래 저장 주기가 300초가 맞습니까? 1분 파일은 채워 넣은 값입니까?	5행마다만 값이 바뀜
HYD01_PRESS_IN	210 bar가 센서 측정 상한입니까, 아니면 시스템 설정 압력입니까?	최댓값 = range_max
미제공 태그 11개	진동 3축·회전수·유량·2존 온도를 함께 받을 수 있습니까?	사양서에만 존재
MTR01_TEMP	09:20 이후 54.0 고정이 실제 안정화입니까, 센서	40행 연속 동일

대상	확인 질문	근거
	고착입니까?	
전 태그	센서 교체·교정 이력이 이 2시간 안에 있었습니까?	추세 해석의 전제

설비 적용

질문 목록의 가치는 "모른다"를 "무엇을 물어야 한다"로 바꾼다는 데 있습니다. 회의에 "데이터가 이상합니다"를 들고 가면 대화가 안 되지만, "FUR01_TEMP_Z1이 5행마다만 바뀌는데 원주기가 300초인지 확인 부탁드립니다"를 들고 가면 그 자리에서 답이 나옵니다. 근거 열이 붙어 있는 것이 특히 중요합니다.

5-7. 실습 1 전체를 한 스크립트로

셀 단위로 흩어져 있던 과정을 하나로 이어 붙이면, 새 데이터를 받을 때마다 그대로 돌릴 수 있는 점검 스크립트가 됩니다. 오늘 배운 것을 재사용 가능한 형태로 남기는 방법입니다.

점검 스크립트

```
import pandas as pd

TAGS_PATH = "02-01_측정의_3요소_설비태그목록.csv"
SAMPLE_PATH = "02-01_측정의_3요소_측정샘플.csv"

tags = pd.read_csv(TAGS_PATH, encoding="utf-8")
df = pd.read_csv(SAMPLE_PATH, encoding="utf-8")
df["timestamp"] = pd.to_datetime(df["timestamp"])
cols = [c for c in df.columns if c != "timestamp"]

# ① 실제 저장 간격
gap = df["timestamp"].diff().dt.total_seconds().dropna()
real_sec = int(gap.mode()[0])
print(f"실제 저장 간격 {real_sec}초 · 등간격 여부 {gap.nunique() == 1}")

# ② 컬럼별 점검
spec = tags.set_index("tag")
for c in cols:
    step = df[c].diff().abs().replace(0, pd.NA).min()
    block = (df[c] != df[c].shift()).cumsum()
    rep = df.groupby(block)[c].size().max()
    hit = (df[c] == spec.loc[c, "range_max"]).sum()
    flags = []

    if spec.loc[c, "sampling_sec"] != real_sec: flags.append("주기불일치")
    if hit > 0: flags.append(f"상한{hit}회")
    if abs(step - spec.loc[c, "resolution"]) < 1e-9: flags.append("분해능일치")
    print(f"{c:16} 반복{rep:3} 최소폭 {round(step,4):>5} | " + " ".join(flags))
```

- ▶ 실제 저장 간격 60초 · 등간격 여부 True
- ▶ MTR01_VIB_RMS_H 반복 2 최소폭 0.1 | 분해능일치
- ▶ MTR01_CURRENT 반복 1 최소폭 0.5 | 분해능일치
- ▶ MTR01_TEMP 반복 40 최소폭 0.5 | 분해능일치
- ▶ HYD01_PRESS_IN 반복 7 최소폭 1.0 | 상한14회 분해능일치
- ▶ FUR01_TEMP_Z1 반복 5 최소폭 0.5 | 주기불일치 분해능일치

출력 마지막 두 줄에 오늘의 결론이 그대로 찍힙니다. HYD01_PRESS_IN에 상한14회, FUR01_TEMP_Z1에 주기불일치. 사람이 표를 눈으로 훑어서 찾아낸 것을 코드가 자동으로 표시해 준 것입니다.

응용

`.mode()[0]`으로 **최빈 간격**을 실제 주기로 삼은 것이 핵심입니다. `.min()`이나 `.max()`를 쓰면 결측 구간 하나에 값이 흔들리지만, 최빈값은 흔들리지 않습니다. 등간격 여부는 `gap.nunique() == 1`로 따로 표시했으니 두 정보가 분리됩니다.

주의

`abs(step - resolution) < 1e-9`로 비교한 것도 의도적입니다. `step == resolution`으로 쓰면 MTR01_VIB_RMS_H가 부동소수점 오차 때문에 `False`가 되어, 정상인 컬럼이 이상으로 표시됩니다. 실수 비교는 항상 허용 오차를 두고 하셔야 합니다.

측정값의 한계

3요소를 알아도 보이지 않는 값이 있습니다

6-1. 시간축 한계와 크기축 한계

CH 4의 샘플링 주기가 **시간축의 한계**였다면, 이번 장은 **크기축의 한계**입니다. 두 축이 나란히 놓여야 그림이 완성됩니다.

축	한계를 정하는 것	놓치는 것	오늘 데이터 예
시간축	샘플링 주기	주기보다 짧은 사건	FUR01_TEMP_Z1의 5분 내 변화
크기축 (아래)	분해능	분해능보다 작은 변화	MTR01_TEMP의 0.5°C 미만 변화
크기축 (위)	측정 상한	상한을 넘는 변화	HYD01_PRESS_IN의 210 초과분

개념

센서가 표현할 수 있는 범위를 벗어난 변화는 데이터에 남지 않습니다. 너무 작으면 값이 아예 움직이지 않고, 너무 크면 값이 상한에서 멈춥니다. 둘 다 **결측으로 표시되지 않기 때문에** `isna()`로는 절대 못 찾습니다. 28일차에 `asfreq()`로 숨은 결측을 드러냈던 것처럼, 여기도 별도의 확인 절차가 필요합니다.

6-2. 분해능 — 너무 작은 변화

개념 분해능 (resolution) 센서가 구분할 수 있는 가장 작은 차이

정의 서로 떨어져 있는 두 값을 서로 다른 값으로 구별해 낼 수 있는 능력입니다. 이보다 작은 차이는 **같은 값으로** 기록됩니다.

```
# 분해능이 0.5인 온도 센서가 실제 온도를 어떻게 기록하는가
실제 = [52.0, 52.2, 52.3, 52.4, 52.5, 52.7]
기록 = [round(t * 2) / 2 for t in 실제]    # 0.5 단위로 반올림

print("실제:", 실제)
print("기록:", 기록)
```

```
▶ 실제: [52.0, 52.2, 52.3, 52.4, 52.5, 52.7]
▶ 기록: [52.0, 52.0, 52.5, 52.5, 52.5, 52.5]
```

왜 쓰나 실제로는 여섯 번 다 다른 값이었는데 기록에는 **두 개의 값**만 남았습니다. 0.2°C씩 꾸준히 오르는 추세가 데이터에서는 **계단**으로 보입니다.

응용 그래서 분해능을 알면 **어떤 분석이 불가능한지**를 미리 알 수 있습니다. 오늘 데이터의 다섯 컬럼에 대해 계산해 보면 이렇습니다.

```
for col in cols:
    res = tags.set_index("tag").loc[col, "resolution"]
    rng = df[col].max() - df[col].min()

    print(f"{col:16} 분해능 {res:>4} | 관측 범위 {rng:>6.1f} | 구분 가능 단계
    {int(rng/res):>3}단계")
```

```
▶ MTR01_VIB_RMS_H   분해능 0.1 | 관측 범위 0.7 | 구분 가능 단계 6단계
▶ MTR01_CURRENT     분해능 0.5 | 관측 범위 4.0 | 구분 가능 단계 8단계
▶ MTR01_TEMP        분해능 0.5 | 관측 범위 6.0 | 구분 가능 단계 12단계
▶ HYD01_PRESS_IN    분해능 1.0 | 관측 범위 59.0 | 구분 가능 단계 59단계
▶ FUR01_TEMP_Z1     분해능 0.5 | 관측 범위 11.5 | 구분 가능 단계 23단계
```

개념

MTR01_VIB_RMS_H는 관측 범위 전체가 **일곱 칸**뿐이고 실제 고유값도 8개입니다. 이 컬럼으로 "진동이 3% 증가했다" 같은 미세한 판단을 하는 것은 불가능합니다. **한 칸 움직이면 약 5% 변화**이기 때문입니다. 참고로 위 출력이 7이 아니라 6인 것은 0.7 / 0.1이 부동소수점에서 6.999...가 되어 int()가 6으로 잘랐기 때문입니다 — 계산 자체가 오차를 만드는 좋은 예입니다.

자주 하는 실수

분해능이 나쁜 컬럼에 rolling().mean()을 걸면 소수점이 생겨 **정밀해진 것처럼 보입니다**. 하지만 없던 정보가 생긴 것이 아니라 계단 사이를 이어 그린 것뿐입니다. 평활 결과를 원본보다 정밀한 값으로 착각하면 안 됩니다.

6-3. 측정 상한 — 너무 큰 변화

개념 측정 상한 (range_max) 센서가 표현할 수 있는 최댓값

정의 실제 값이 이보다 커도 데이터에는 **상한값에서 멈춰서** 기록됩니다. 잘린 흔적이 따로 남지 않습니다.

```
rmax = tags.set_index("tag").loc["HYD01_PRESS_IN", "range_max"]
hit = df["HYD01_PRESS_IN"] == rmax
```

```
print("range_max :", rmax)
print("최댓값 :", df["HYD01_PRESS_IN"].max())
print("상한 도달 :", hit.sum(), "회 /", len(df), "행")
```

```
▶ range_max : 210
▶ 최댓값 : 210.0
▶ 상한 도달 : 14 회 / 120 행
```

왜 쓰나 최댓값이 range_max와 정확히 같고, 그 값이 여러 번 반복된다면 **상한 포화를 의심해야** 합니다. 우연히 딱 210.0이 열네 번 나올 확률은 매우 낮습니다.

응용 값의 분포를 보면 더 분명해집니다. 151~154 사이에 106개가 몰려 있고, 그 사이는 텅 비었다가 **210에 14개**가 따로 서 있습니다. 연속적인 분포가 아닙니다.

```
print(df["HYD01_PRESS_IN"].value_counts().sort_index().to_string())
```

```
▶ HYD01_PRESS_IN
▶ 151.0      21
▶ 152.0      43
▶ 153.0      32
▶ 154.0      10
▶ 210.0      14
```

개념

155부터 209까지는 **단 한 번도 나오지 않았습니다**. 압력이 154에서 210으로 한 번에 점프했다는 뜻인데, 물리적으로는 자연스럽지 않습니다. **잘린 값이거나, 아예 다른 운전 모드일** 가능성이 큼니다.

주의

상한 포화가 의심되면 그 구간을 **분석에서 제외하거나 별도로 표시**해야 합니다. 그대로 평균에 넣으면 실제보다 낮은 값이 되고(실제는 210 이상이었을 수 있으므로), 최댓값 추세를 그리면 상한선을 따라 평평한 직선이 나옵니다.

6-4. 측정값과 참값 사이의 오차

분해능과 상한을 다 통과해도 남는 문제가 있습니다. **측정값은 참값 그 자체가 아닙니다**.

개념

온도가 45.0으로 기록되었다고 해서 실제 온도가 정확히 45.0°C인 것이 아닙니다. **45.2°C일 수도, 44.8°C일 수도 있습니다.** 측정값은 **참값 주변의 추정치**이고, 그 오차 범위는 센서 사양서에 적혀 있습니다.

그래서 다음 규칙이 나옵니다. **오차 범위보다 작은 차이를 해석하면 잡음을 해석하는 것입니다.**

상황	판단	이유
센서 오차 $\pm 1.0^{\circ}\text{C}$, 두 시점 차이 0.4°C	해석 불가	오차 범위 안 — 같은 값일 수 있음
센서 오차 $\pm 1.0^{\circ}\text{C}$, 두 시점 차이 5.0°C	해석 가능	오차보다 충분히 큼
센서 오차 $\pm 1.0^{\circ}\text{C}$, 100개 평균의 차이 0.4°C	조건부 가능	평균은 랜덤 오차가 줄어들

주의

세 번째 행이 중요합니다. **랜덤 오차는 평균을 내면 줄어들지만, 계통 오차(치우침)는 평균을 내도 줄어들지 않습니다.** 센서가 항상 1°C 높게 읽는다면 백만 번 평균을 내도 1°C 높습니다. 그래서 **절대값 비교보다 같은 센서 안에서의 변화 비교**가 안전합니다 — 계통 오차가 상쇄되기 때문입니다.

6-5. "0.5°C보다 작은 변화는 데이터에 없다"

교안이 슬라이드 한 장을 통째로 써서 남긴 문장입니다. 오늘 배운 것 전체를 한 줄로 압축한 셈입니다.

개념

"데이터에 없다"와 "실제로 없었다"는 다릅니다. 0.3°C 올랐어도 분해능 0.5°C 센서에는 안 나타납니다. 30초짜리 스파이크가 있었어도 1분 주기 저장에는 안 남습니다. 215 bar까지 올라갔어도 상한 210 센서에는 210으로 기록됩니다. **셋 다 데이터를 아무리 잘 분석해도 복원되지 않습니다.**

이 인식이 실무에서 바꾸는 것은 두 가지입니다.

상황	인식이 없을 때	인식이 있을 때
이상이 안 잡힘	"이상이 없었다"고 보고	"이 해상도로는 못 본다"고 보고
프로젝트 초기	받은 데이터로 최선을 다함	필요한 해상도를 먼저 협의
모델 성능이 낮음	알고리즘을 바꿔 봄	데이터의 관찰 한계부터 확인
값이 안 변함	고장 난 센서로 의심	분해능·상한부터 대조

설비 적용

모델이 안 나올 때 알고리즘부터 바꾸는 것은 대개 헛수고입니다. 분해능 0.1 mm/s로 7단계밖에 안 되는 진동 값에 덤러닝을 붙여도, 없는 정보가 생기지는 않습니다. 오늘 배운 순서는 ① 관찰 한계 확인 → ② 필요한 해상도 협의 → ③ 그다음 모델입니다.

6-6. 센서 카드 — 컬럼 하나를 한 장으로

오늘 배운 것을 실제 업무 산출물로 만드는 형식입니다. 컬럼 하나마다 이 카드를 채우면 데이터 사전이 완성됩니다.

항목	작성 내용	예시
태그명	원본 컬럼명 그대로	MTR01_VIB_RMS_H
물리량	측정 대상	진동 속도
단위	접두어·압력 기준까지 명시	mm/s
설치 위치	설비 · 지점 · 방향	1번 모터 구동측 수평

교안이 제시한 네 항목에, 오늘 실습에서 확인한 것들을 더하면 훨씬 쓸모 있는 카드가 됩니다.

추가 항목	오늘 확인한 값	확인 방법
사양 주기 / 실제 간격	60초 / 60초	<code>timestamp.diff()</code>
분해능	0.1 mm/s	최소 변화 폭 = resolution
측정 범위	0 ~ 25 mm/s	<code>range_min ~ range_max</code>
관측 범위 / 단계 수	1.7 ~ 2.4 (7단계)	실제 데이터
값의 성격	직접 측정값	설치 위치에 "계산값" 없음
남은 질문	비구동측 값도 받을 수 있는가	미제공 태그 대조

설비 적용

이 카드를 컬럼 수만큼 만드는 것이 데이터 사전 구축입니다. 지루해 보이지만, 프로젝트 후반에 "이 값이 뭘었더라"로 돌아오는 시간을 통째로 없애 줍니다. 오늘 태그 목록 CSV가 바로 잘 만들어진 데이터 사전의 예시이고, 여기에 실제 데이터에서 확인한 열을 덧붙이는 것이 우리가 할 일입니다.

실습 1의 최종 산출물

태그 목록 + 실제 데이터 확인 결과를 합쳐 센서 카드 표 만들기

```
card = tags.set_index("tag").loc[cols, ["description", "unit", "sampling_sec",
                                         "range_max", "resolution",
                                         "install_location"]].copy()
card["실제간격"] = 60
card["실제최소"] = df[cols].min().values
card["실제최대"] = df[cols].max().values
card["최소변화폭"] = [round(df[c].diff().abs().replace(0, pd.NA).min(), 4) for c in cols]
card["주기일치"] = card["sampling_sec"] == card["실제간격"]
card["상한도달"] = [(df[c] == card.loc[c, "range_max"]).sum() for c in cols]

print(card[["sampling_sec", "실제간격", "주기일치", "최소변화폭",
            "resolution", "상한도달"]].to_string())
```

	sampling_sec	실제간격	주기일치	최소변화폭	resolution	상한도달
▶ MTR01_VIB_RMS_H	60	60	True	0.1	0.1	0
▶ MTR01_CURRENT	60	60	True	0.5	0.5	0
▶ MTR01_TEMP	60	60	True	0.5	0.5	0
▶ HYD01_PRESS_IN	60	60	True	1.0	1.0	14
▶ FUR01_TEMP_Z1	300	60	False	0.5	0.5	0

마지막 표의 주기일치 열에 False 하나, 상한도달 열에 14 하나. 오늘 두 시간 실습의 결론이 이 두 칸에 들어 있습니다.

치트시트

오늘 나온 표를 한 장으로

A-1. 설비 4대 분류 — 한 줄 요약

분류	한 줄	첫 관측값	분석 첫 단계
회전기계	축이 돈다. 베어링이 먼저 상한다	진동·전류·온도	바로 추세 분석 가능
유압·공압	기름으로 민다. 명령과 실제를 비교한다	압력·유량·유온	동작 구간 선별
열설비	데운다. 조작량이 먼저 움직인다	온도·연료유량·노압	지연 보정·존별 분포
반송	움긴다. 조건이 값을 바꾼다	전류·속도·위치	조건 구간 선별 + 부하 보정

A-2. 측정의 3요소 체크리스트

요소	확인할 것	데이터로 확인하는 방법
물리량	단위·description·값의 범위	<code>agg(["min", "max"]) + range_min/max</code> 대조
설치 위치	<code>install_location</code> ·"계산값" 여부	파생값이면 원본 두 개의 정밀도 확인
샘플링 주기	사양 주기 vs 실제 저장 간격	<code>timestamp.diff().value_counts()</code>
(추가) 분해능	최소 변화 폭	<code>.diff().abs().replace(0, pd.NA).min()</code>
(추가) 측정 상한	최댓값 반복 여부	<code>(s == range_max).sum()</code>

A-3. 오늘 쓴 pandas 도구

분류	이름	하는 일
함수	<code>pd.read_csv()</code>	CSV를 DataFrame으로 읽기
함수	<code>pd.to_datetime()</code>	문자열을 시간 타입으로 변환
속성	<code>.columns</code>	컬럼 이름 목록 (괄호 없음)
속성	<code>.dtype / .shape</code>	자료형 / (행, 열) 크기
메서드	<code>.str.startswith()</code>	접두어 검사 — 여러 개는 튜플
메서드	<code>.rename(columns={...})</code>	컬럼 이름 바꾸기 (새 객체 반환)
메서드	<code>.diff()</code>	앞 행과의 차이

분류	이름	하는 일
메서드	<code>.abs()</code>	절댓값
메서드	<code>.replace(0, pd.NA)</code>	특정 값을 결측으로 바꾸기
메서드	<code>.value_counts(dropna=False)</code>	값별 개수 — 결측도 포함
메서드	<code>.nunique()</code>	고유값 개수
메서드	<code>.agg(["min", "max", "mean", "std", ""])</code>	여러 통계를 한 번에
메서드	<code>.set_index("tag")</code>	특정 열을 인덱스로 — 조회가 쉬워짐
메서드	<code>.groupby(block).size()</code>	블록별 크기 세기
패턴	<code>(s != s.shift()).cumsum()</code>	값이 바뀔 때마다 번호가 오르는 블록 ID

A-4. 판정 규칙 요약

관찰된 현상	먼저 확인할 것	판정
같은 값이 N행마다 반복	사양 주기 ÷ 실제 간격 = N ?	주기 맞춤 반복 (정보 없음)
최댓값 = range_max, 여러 번 반복	값 분포에 빈 구간이 있는가	측정 상한 포화 의심
최댓값 ≪ range_max, 오래 반복	물리적으로 안정될 상황인가	실제 안정화 가능성
최소 변화 폭 = resolution	—	정상 — 분해능 한계
값이 계단으로만 움직임	분해능 때문인가 주기 때문인가	둘 다 가능 — 구분 필요
온도가 한 스텝에 급등	물리적으로 가능한 속도인가	센서 이상 후보

개념

다섯 번째 행이 헛갈리기 쉽습니다. 분해능 때문인 계단은 변화 폭이 항상 분해능의 정수 배이고, 주기 때문인 계단은 값이 바뀌는 간격이 일정합니다. MTR01_TEMP는 전자(0.5씩 움직이지만 간격은 불규칙), FUR01_TEMP_Z1은 후자(정확히 5행마다)입니다. 변화 폭을 볼지 변화 간격을 볼지가 판별의 갈림길입니다.

A-5. 오늘 데이터의 최종 요약

태그	물리량	단위	평균	최소~최대	분해능	주기(사양/실제)
MTR01_VIB_RMS_H	진동 속도	mm/s	2.020	1.7 ~ 2.4	0.1	60 / 60
MTR01_CURRENT	운전 전류	A	63.750	62 ~ 66	0.5	60 / 60

태그	물리량	단위	평균	최소~최대	분해능	주기(사양/실제)
MTR01_TEMP	권선 온도	degC	52.762	48 ~ 54	0.5	60 / 60
HYD01_PRESS_IN	공급 압력	bar	159.025	151 ~ 210	1.0	60 / 60
FUR01_TEMP_Z1	1존 온도	degC	1185.750	1180 ~ 1191.5	0.5	300 / 60

표준편차는 순서대로 **0.187 · 1.083 · 1.553 · 18.622 · 3.476**입니다. HYD01_PRESS_IN만 유독 큰 이유는 210 구간 14개가 평균에서 멀리 떨어져 있기 때문이고, 그것이 곧 상한 포화 의심의 출발점이었습니다.

A-6. 태그 약어 사전 (배포 자료 24개)

약어	뜻	약어	뜻
MTR	Motor — 모터	PRESS	Pressure — 압력
HYD	Hydraulic — 유압	DP	Differential Pressure — 차압
FUR	Furnace — 가열로	FLOW	유량
VIB	Vibration — 진동	OILTEMP	Oil Temperature — 작동유 온도
RMS	Root Mean Square — 실효값	H / V / A	수평 / 수직 / 축방향
ACC	Acceleration — 가속도	IN / OUT	입구(공급) / 출구(후단)
CURRENT	전류	FILTER	필터
VOLT	전압	Z1 / Z2	1존 / 2존
POWER	전력	01	설비 호기 번호
TEMP	Temperature — 온도	RPM	Revolutions Per Minute — 회전수

용어·오류 사전

워크시트 교정 · 노트북 코드 버그 · 헛갈리는 용어

B-1. 실습 워크시트 교정 — CASE A

제출하신 워크시트의 **CASE B**는 태그 목록과 완전히 일치했고 **Step 3**의 통계 표도 다섯 줄 전부 정확했습니다. CASE A에서만 다섯 군데를 바로잡으면 됩니다.

태그	칸			이유
MTR01_VIB_RMS_H/V/A	물리	회전	진동 (진동 속도)	단위가 mm/s — 속도
MTR01_VIB_ACC	물리	무게	진동 가속도	g는 중력가속도 단위
MTR01_RPM	물리	회전	회전수	단위 rpm — 개수/시간
MTR01_RPM	설치 위치	측정 회전수 센서	측단 회전수 센서	태그 목록 원문
V · A · ACC 행	설치 위치	구동축 / 비구동축	구동축 / 비구동축	축(shaft)이 아니라 측(side)

개념

"구동축"과 "구동축"은 완전히 다른 말입니다. 구동축(drive end)은 모터에서 부하가 연결된 쪽이라는 *위치*를 뜻하고, 구동축은 *부품*을 뜻합니다. 설치 위치 칸에 들어가야 하는 것은 위치이므로 "측"이 맞습니다. 첫 줄 (MTR01_VIB_RMS_H)에는 "구동축"으로 정확히 쓰셨는데 아래 줄부터 "측"으로 바뀌었으니, 옮겨 적으면서 생긴 오타로 보입니다.

주의

g를 무게로 읽으신 것은 흔한 착각입니다. 진동 가속도의 g는 중력가속도(9.8 m/s^2)를 1로 놓은 단위입니다. "3 g"는 중력의 세 배 가속도라는 뜻이지 3그램이 아닙니다. 4-4절의 세 단위 표(mm/s · g · μm)를 함께 외워 두시면 헛갈리지 않습니다.

B-2. 실습 워크시트 교정 — Step 3 마지막 질문

"값이 계단처럼 변하는 태그는?"에 MTR01_TEMP라고 답하셨습니다. 맞는 답이지만 하나가 더 있습니다.

태그	계단인가	계단이 생긴 이유	성격
MTR01_TEMP	예	분해능 0.5 — 값이 0.5 단위로만 움직임	크기축 한계
FUR01_TEMP_Z1	예 (누락)	사양 주기 300초 — 5행마다만 값이 바뀜	시간축 한계

개념

두 계단은 **원인이 완전히 다릅니다**. MTR01_TEMP는 값의 **높이**가 0.5씩 뛰는 계단이고(변화 간격은 불규칙), FUR01_TEMP_Z1은 값의 **너비**가 5행씩인 계단입니다(변화 폭은 0.5~1.0으로 다양). **높이 계단은 분해능, 너비 계단은 샘플링 주기**입니다. 이 구분이 오늘 실습 체크포인트 ①과 ②를 가르는 지점입니다.

B-3. 배포 노트북의 코드 버그

02-01_example.ipynb 마지막 셀에서 **MTR 태그의 분해능이 전부 None으로 출력됩니다. 조회 대상 DataFrame을 잘못 잡았기 때문**입니다.

실행 결과 — 3개가 None

```
# x 원래 코드
res = df_hyd_fur.set_index("tag")["resolution"].get(col)
```

```
# df_hyd_fur 에는 HYD·FUR 7개만 들어 있는데,
# 반복문은 MTR 컬럼까지 조회하므로 MTR은 전부 None
```

```
▶ MTR01_VIB_RMS_H   분해능 None
▶ MTR01_CURRENT     분해능 None
▶ MTR01_TEMP        분해능 None
▶ HYD01_PRESS_IN    분해능 1.0
▶ FUR01_TEMP_Z1     분해능 0.5
```

수정 후

```
# ✓ 바로잡기 — 전체 태그 목록(tags)에서 조회
res = tags.set_index("tag")["resolution"].get(col)
```

```
▶ MTR01_VIB_RMS_H   분해능 0.1
▶ MTR01_CURRENT     분해능 0.5
▶ MTR01_TEMP        분해능 0.5
▶ HYD01_PRESS_IN    분해능 1.0
▶ FUR01_TEMP_Z1     분해능 0.5
```

자주 하는 실수

`.get()`은 키가 없으면 오류 대신 `None`을 돌려줍니다. 편리하지만 위험합니다 — 조회가 실패해도 코드가 멈추지 않아서, 세 줄이 `None`인 채로 넘어갑니다. 조회 실패를 놓치고 싶지 않을 때는 `.get()` 대신 `[col]`로 직접 접근해 `KeyError`가 나게 두는 편이 낫습니다.

이 버그의 아이러니는, **버그 덕분에 오늘 실습의 핵심을 놓칠 뻔했다**는 점입니다. MTR 세 개의 최소 변화 폭이 분해능과 정확히 일치하는지 확인하는 것이 실습의 목표였는데, 대조 대상이 `None`이면 확인 자체가 안 됩니다.

B-4. 노트북에 남은 다른 지점들

위치			설명
셀 10 주석	<code>ts.columns()</code>	<code>ts.columns</code>	속성에 괄호를 붙이면 <code>TypeError</code>
셀 8	<code>round(..., 4)</code> 로 0.1 표시	그대로 두되 비교는 반올림 후	부동소수점 오차 0.09999...
셀 10~11	<code>df.info()</code> 가 태그 목록을 가리킴	변수명을 명확히 분리	같은 셀에 <code>ts</code> 와 <code>df</code> 가 혼재
셀 2	<code>DATA_PATH</code> 에 태그 목록 경로	—	측정 샘플과 헷갈리지 않게 주의

주의

세 번째 행이 실제로 헷갈립니다. 노트북에서 `df`는 **태그 목록**이고 `df_ts`가 **측정 샘플**인데, 셀 10에서는 측정 샘플을 `ts`라는 새 이름으로 다시 읽습니다. 변수가 세 개(`df`, `df_ts`, `ts`)로 늘어나면 어느 것이 무엇인지 흐려집니다. `tags`와 `sample`처럼 내용을 담은 이름으로 통일하시는 편이 안전합니다.

B-5. 오늘 헷갈리기 쉬운 용어 정리

용어 A	용어 B	차이
구동측 (drive end)	비구동측 (non-drive end)	부하가 연결된 쪽 / 반대쪽. 같은 고장도 값이 다름
분해능	정확도	구분 가능한 최소 폭 / 참값과의 거리. 별개의 개념
사양 주기	실제 저장 간격	사양서의 값 / 타임스탬프로 확인한 값
직접 측정값	파생값	센서가 낸 값 / 다른 값으로 계산한 값
제어 대상	조작량	목표를 유지할 값 / 목표를 위해 바꾸는 값
원인 지시형	결과형	원인을 직접 짚음 / 여러 원인이 겹쳐 나타남
진동 속도 (mm/s)	진동 가속도 (g)	불평형·정렬불량 / 베어링 초기 손상

용어 A	용어 B	차이
상태값	부하값	장기 추세용 분 단위 / 조건 비교용 초 단위

개념

분해능과 정확도는 특히 자주 섞입니다. 분해능이 0.1인 센서가 항상 5도씩 높게 읽을 수도 있습니다 — 이 경우 **잘게 구분은 하지만 다 틀린 값**입니다. 반대로 정확도는 좋은데 분해능이 나쁘면 **맞긴 맞는데 뭉뚱한 값**입니다. 추세를 볼 때는 분해능이, 절대값을 볼 때는 정확도가 중요합니다.

하이라이트와 다음 시간

오늘의 핵심 발견 · 31일차 예고

C-1. 오늘의 발견 세 가지

① 사양서가 틀린 것이 아니라, 저장 방식이 다른 것입니다

FUR01_TEMP_Z1의 사양 주기는 300초인데 파일의 간격은 60초입니다. 처음에는 "사양서가 틀렸나" 싶지만, **값이 정확히 5행마다 바뀌는 것**을 보면 답이 나옵니다. 원래 5분에 한 번 읽는 센서인데, 저장 시스템이 1분 표에 맞춰 같은 값을 다섯 번 채워 넣은 것입니다.

확인	값	의미
사양 주기	300초	센서가 실제로 읽는 간격
저장 간격	60초	파일에 행이 만들어지는 간격
값 변경 간격	5행 = 300초	실제 정보가 갱신되는 간격
총 변경 횟수	23회 / 120행	2시간 동안 유효 관측 24개

설비 적용

이 컬럼으로 1분 단위 분석을 하면 결과의 80%가 허구입니다. `.diff()`를 걸면 5행 중 4행이 0이 되고, `rolling(10).std()`를 걸면 실제보다 작은 변동성이 나옵니다. **5분으로 리샘플링해서 24개 관측으로 줄여 놓고 봐야** 정직한 분석입니다.

② 같은 "값이 안 변한다"도 원인이 셋입니다

컬럼	현상	원인	대응
FUR01_TEMP_Z1	5행씩 동일	샘플링 주기 (시간축)	리샘플링
MTR01_TEMP	09:20 이후 40행 동일	실제 안정화 (물리)	정상 — 그대로 분석
HYD01_PRESS_IN	210이 7행씩 두 번	측정 상한 의심 (크기축)	현장 확인 후 판단

셋 다 `isna()`로는 안 잡히고 셋 다 "값이 반복된다"로 보입니다. **원인을 가르는 것은 사양서와의 대조뿐**입니다.

`resolution`, `sampling_sec`, `range_max` — 이 세 열이 없으면 구분이 불가능합니다.

③ 데이터의 절반은 애초에 오지 않았습니다

태그 목록 16개 중 실제로 받은 것은 5개입니다. 진동 3축 중 수평 하나만, 회전수는 없고, 2존 온도도 없습니다. 1-5절의 진동 반복 주기 진단은 회전수 없이는 시작조차 못 하고, 3-3절의 준별 편차는 2존 없이는 계산이 안 됩니다.

하고 싶은 분석	필요한 태그	현재 상태
진동 이상 원인 판별	MTR01_RPM + 진동 3축	불가 — 회전수·V·A 없음
베어링 초기 손상 감지	MTR01_VIB_ACC	불가 — 가속도 없음
필터 막힘 판정	HYD01_DP_FILTER	불가 — 차압 없음
가열로 국부 열화	FUR01_TEMP_Z2	불가 — 2존 없음
모터 부하 추세	전류 · 온도	가능

설비 적용

이 표가 그대로 현장 협의 자료가 됩니다. "데이터를 더 주세요"가 아니라 "베어링 초기 손상을 보려면 MTR01_VIB_ACC가 필요합니다"라고 말하면, 요청의 물리적 근거가 붙어서 협의가 훨씬 빨라집니다. 교안 39쪽이 강조한 지점이 정확히 이것입니다.

C-2. 저녁에 해 보면 좋을 것

난이도	과제	확인 포인트
쉬움	워크시트 CASE A의 다섯 군데를 고쳐 다시 채우기	B-1의 교정표
쉬움	노트북 셀 9의 df_hyd_fur를 tags로 바꿔 실행	MTR 세 개의 분해능이 나오는지
보통	FUR01_TEMP_Z1을 5분으로 리샘플링해 관측 24개로 줄이기	<code>resample("5min").first()</code>
보통	210 구간 두 개를 제외하고 압력 통계를 다시 내기	평균과 표준편차가 얼마나 바뀌는지
도전	다섯 컬럼의 센서 카드를 6-6절 형식으로 완성하기	남은 질문 칸까지 채우기

주의

세 번째 과제에서 `resample("5min").mean()`이 아니라 `.first()`를 쓰셔야 합니다. 어차피 5행이 같은 값이라 평균도 같은 값이 나오지만, 평균은 "여러 관측을 합쳤다"는 잘못된 인상을 줍니다. 실제로는 관측이 하나뿐이니 `.first()`가 의미에 맞습니다.

C-3. 다음 시간 예고 — 31일차

오늘까지가 02-01 설비 4대 분류 한 편이었습니다. 다음은 오늘 만든 틀 위에 신호 하나하나를 엮는 순서입니다.

교안	주제	오늘과의 연결
03-01	기계 설비 신호	오늘 1장의 회전기계·유압·열설비를 신호 단위로 깊게
03-02	센서와 센서 고장	오늘 6장의 "급격한 도약은 센서 이상 후보"를 판별법으로

실습도 늘어납니다. **실습 5(회전기계 신호) · 실습 6(유압 계통) · 실습 7(열설비) · 실습 8(센서 고장 판별)**이 이어질 예정이고, 오늘의 4대 분류가 그 네 실습의 뼈대가 됩니다.

연결

오늘 배운 **측정의 3요소와 관찰 한계**는 다음 시간에도 계속 쓰입니다. 센서 고장을 판별한다는 것은 결국 **"이 값이 물리적으로 가능한가"**를 묻는 일이고, 그 판단 기준이 오늘의 분해능·측정 상한·응답 속도입니다. 6장을 한 번 더 읽고 가시면 다음 시간이 훨씬 편합니다.

EPILOGUE

맺음말

오늘의 성취와 복습 루틴

오늘 하신 것

구간	성취
전반부	수천 개 설비 이름을 네 가지 동작 방식으로 압축하는 틀을 익혔습니다
전반부	분류마다 첫 분석 단계가 다른 이유를 데이터 특징으로 설명할 수 있게 되었습니다
후반부	컬럼 하나를 물리량·설치 위치·샘플링 주기로 분해하는 습관을 만들었습니다
실습	사양서와 실제 기록을 대조해 주기 불일치와 상한 반복을 직접 찾아냈습니다
마무리	데이터로 못 채운 칸을 현장 질문으로 바꾸는 방법을 익혔습니다

복습 루틴 제안

시점	할 것	걸리는 시간
오늘 밤	A-1 · A-2 치트시트 두 표만 다시 읽기	5분
내일 아침	B-1 교정표를 보고 워크시트 CASE A 다시 채우기	10분
주말	C-2의 보통·도전 과제 실행 + 센서 카드 완성	1시간
다음 주	6장(측정값의 한계)을 03-02 센서 고장과 함께 다시 읽기	20분

주의

오늘 자료 중 **가장 오래 쓰이는 것은 A-2 체크리스트**입니다. 새 데이터를 받을 때마다 그 다섯 줄을 순서대로 돌리면, 분석을 시작하기 전에 **이 데이터로 무엇이 가능하고 무엇이 불가능한지**가 먼저 정해집니다. 그 순서를 지키는 것과 지키지 않는 것의 차이가 프로젝트 후반에 몇 주 차이로 돌아옵니다.

오늘 배운 문장 하나만 가져가신다면 이것입니다. "**데이터에 없다**"와 "**실제로 없었다**"는 다릅니다. 분해능보다 작은 변화, 주기보다 짧은 사건, 상한을 넘는 값 — 셋 다 아무리 잘 분석해도 복원되지 않습니다. 분석가가 가장 먼저 확인해야 하는 것은 알고리즘이 아니라 **자기가 받은 데이터의 관찰 한계**입니다.

수고하셨습니다.